

AI-Based Face Recognition Attendance System

By

Azeem Aslam

Reg # 2022-GCUF-06887

Ayesha Shahid

Reg # 2022-GCUF-06907

BACHELOR OF SCIENCE
IN
INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD.

June 2026

DECLARATION

The work reported in this documentation was carried out by us under the supervision of **[Dr. Mariam Rehman]** Department of Information Technology Government College University, Faisalabad, Pakistan.

We hereby declare that the title of project **[AI-Based Face Recognition Attendance System]** and the contents of the documentation are the product of my own work and no part has been copied from any published source (except the references, standard mathematical or genetic models /equations /formulas/protocols etc.) we further declare that this work has not been submitted for award of any other degree /diploma. The University may take action if the information provided is found inaccurate at any stage.

Signature of the Student

Azeem Aslam
2022-GCUF-06887

Signature of the Student

Ayesha Shahid
2022-GCUF-06907

CERTIFICATE BY SUPERVISORY COMMITTEE

We certify that the contents and form of documentation submitted by **Mr. Azeem Aslam** Registration No: **2022-GCUF-06887** and by **Ms. Ayesha Shahid** Registration No: **2022-GCUF-06907**. Has been Found satisfactory and in accordance with the prescribed format. We recommend it to be processed for the evaluation by the External Examiner for the award of Degree.

Signature of Supervisor

Name.....

Designation with stamp.....

Member of Supervisory Committee:

Signature

Name.....

Designation with stamp.....

Member of Supervisory Committee:

Signature

Name.....

Designation with stamp.....

Chairperson:

Signature

Name.....

Designation with stamp.....

DEDICATED TO

We dedicate this project to our respected teachers and supervisor whose guidance, encouragement, and valuable knowledge helped us throughout the completion of this final year project. Their continuous support and patience motivated us to overcome every challenge during this journey.

We would also like to dedicate this project to our parents and families for their endless prayers, love, and support. Their belief in us gave us the confidence and strength to achieve this milestone successfully.

We are also thankful to our friends and classmates who supported, encouraged, and motivated us during different stages of this project and made this journey more memorable.

Lastly, we dedicate this achievement to ourselves as a team of two members for our hard work, dedication, teamwork, and countless efforts that turned this project into a successful accomplishment.

ACKNOWLEDGEMENTS

All praise to Almighty ALLAH, the most merciful and the most compassionate, and His Holy Prophet Muhammad (Peace Be Upon Him), the most perfect and exalted, who is forever a torch of guidance and knowledge for humanity as a whole.

We would like to express our heartfelt gratitude to our supervisor [Dr. Mariam Rehman] for providing invaluable guidance, constructive feedback, and continuous motivation throughout the development of this project.

We are also grateful to the Department of Information Technology, Government College University Faisalabad, for providing the necessary resources and infrastructure required to complete this Final Year Project.

Finally, we thank our families and friends for their moral support and encouragement throughout this academic journey.

Azeem Aslam

Ayesha Shahid

TABLE OF CONTENT

CHAPTER 1: Software Requirement Specification	1
1.1 Introduction.....	1
1.1.1 Project Objectives	1
1.1.2 Scope of the System.....	1
1.1.3 Feasibility Analysis.....	2
1.2 Stakeholders	2
1.3 Users of the System	3
1.4 Functional Requirements	3
1.4.1 List of Functional Requirements.....	3
1.4.2 Description of Each Requirement in Requirement Shell.....	4
1.5 Non-Functional Requirements	5
1.6 Schedule of the Project	6
CHAPTER 2: Analysis	8
2.1 Use Case Model	8
2.1.1 Description of Each Use Case.....	9
2.1.2 System Sequence Diagrams (SSD).....	11
2.1.3 Domain Model	14
CHAPTER 3: Project Design.....	16
3.1 Design Class Diagram.....	16
3.2 Entity Relationship Diagram (ERD).....	17
3.3 Sequence Diagrams.....	18
3.4 System Architecture.....	19
CHAPTER 4: Implementation	20
4.1 Tools & Technologies Used.....	20
4.2 System Modules Description	20
4.3 Code / Algorithm of Main Scenarios	22
4.3.1 Face Recognition Pipeline	22
4.3.2 Attendance Session Flow	22
CHAPTER 5: System Testing.....	35
5.1 Test Cases	35
5.2 Testing Types.....	37

• Unit Testing	37
• Integration Testing	37
• System Testing.....	37
• User Acceptance Testing (UAT)	37
CHAPTER 6: User Interfaces.....	38
6.1 Login & Authentication Screens.....	38
6.2 Admin Dashboard	42
6.3 Teacher Interface	47
6.4 Student Interface	48
6.5 Reports & Audit Logs.....	50

LIST OF FIGURES

Figure 1 Agile Diagram	7
Figure 2 Use Case model	8
Figure 3 Student Registration	12
Figure 4 Teacher Registration.....	13
Figure 5 Teacher Login Verification	13
Figure 6 Live Attendance Session	14
Figure 7 Student Marking Attendance using face recognition	14
Figure 8 Domain Model.....	15
Figure 9 Design Class Diagram	16
Figure 10 Entity Relationship Model.....	17
Figure 11 Sequence Diagram.....	18
Figure 12 System Architecture	19
Figure 13 Main Interface.....	38
Figure 14 Roles in Interface.....	38
Figure 15 How Website Work	39
Figure 16 Features Of Website	39
Figure 17 Login Page.....	40
Figure 18 Teacher Login Verification	40
Figure 19 OTP Verification	41
Figure 20 OTP VIA G Mail	41
Figure 21 Admin Dashboard.....	42
Figure 22 Department Creation.....	42

Figure 23 Departments Created by Admin	43
Figure 24 Section Creation	43
Figure 25 Section Created By Admin	43
Figure 26 Subject Creation	44
Figure 27 Subject Created By Admin	44
Figure 28 Collected Data Samples.....	45
Figure 29 All Datasets.....	45
Figure 30 Dataset Training	46
Figure 31 Trained Models.....	46
Figure 32 Teacher Dashboard	47
Figure 33 Live Attendance Sessions	47
Figure 34 Student Attendance Manage by Teacher	47
Figure 35 Edit Attendance	48
Figure 36 Student Dashboard.....	48
Figure 37 Student Attendance Mark Using Face Recognition	49
Figure 38 Student Attendance History	49
Figure 39 Report Generation.....	50
Figure 40 Report Generated.....	50
Figure 41 System Logs	51

LIST OF TABLES

Table 1 Functional requirement of user registration	4
Table 2 Functional Requirement of Teacher Face Verification	4
Table 3 Functional Requirement of Live Attendance Session	5
Table 4 Functional Requirement of Attendance Report Generation	5
Table 5 Functional Requirement of Audit Log	5
Table 6 Sprint Table	6
Table 7 User Registration Use Case	9
Table 8 Teacher Face Verification Login Use Case	10
Table 9 Live Attendance Session Use Case	11
Table 11 User Registration Test Case	35
Table 12 Teacher Face Recognition Login Test Case	35
Table 13 Student Live Attendance Test Case	36
Table 14 Attendance Record Locking Test Case	36
Table 15 Report Generation Test Case	37

ABSTRACT

The Face Recognition-Based Attendance Management System (FRA System) is a full-stack web application developed to automate and secure student and teacher attendance using Artificial Intelligence. Traditional attendance methods are time-consuming, error-prone, and vulnerable to proxy attendance — a problem where one student marks attendance on behalf of another.

The FRA System addresses these challenges by integrating a robust biometric face recognition engine using OpenCV, HOG+LBP feature extraction, and a Convolutional Neural Network (CNN) trained on student facial datasets. The system offers role-based access for three types of users: Admin, Teacher, and Student. Attendance sessions are initiated by teachers, during which students authenticate themselves via live face scan. Biometric records, once captured, are locked to prevent tampering.

The system is built using Python and Django for the backend, with TensorFlow and scikit-learn supporting the AI pipeline. The frontend uses HTML5, CSS3, Bootstrap, and JavaScript, with WebRTC enabling real-time webcam interaction. Key features include OTP verification, anti-spoofing measures, audit logs with IP and timestamp tracking, automated report generation (daily, weekly, monthly), and a department-section-subject academic hierarchy.

The FRA System provides an efficient, scalable, and tamper-proof solution to attendance management in academic institutions, significantly reducing manual effort while improving accountability and accuracy.

CHAPTER 1: Software Requirement Specification

1.1 Introduction

The Face Recognition-Based Attendance Management System (FRA System) is an AI-driven, full-stack web application designed to revolutionize attendance management in academic institutions. The system leverages biometric face recognition technology to automate the process of recording attendance for both students and teachers, replacing outdated and error-prone manual methods.

In conventional attendance systems, teachers manually call the roll no or students sign sheet both of which are susceptible to human errors, buddy punching (proxy attendance), and inefficiency. The FRA System eliminates these vulnerabilities by using real-time face scanning, deep learning-based identity verification, and a secure role-based access infrastructure accessible through standard web browsers.

1.1.1 Project Objectives

- **General Objectives**

To develop a secure, AI-powered web-based attendance management system that automates attendance recording using facial recognition, supports role-based user management, and provides real-time reports and audit trails for academic institutions.

- **Specific Objectives**

1. Eliminate proxy attendance by requiring biometric face verification for each attendance session.
2. Provide role-based access control for three user types: Admin, Teacher, and Student.
3. Generate accurate and detailed attendance reports on a daily, weekly, and monthly basis.
4. Implement anti-spoofing measures to prevent photo or video-based attacks on the face recognition system.
5. Maintain a complete audit log of all system activities with timestamps and IP addresses.
6. Enable teachers to manage attendance sessions and students to mark attendance via live webcam.
7. Allow administrators to manage department, section, subject, and user hierarchies.

1.1.2 Scope of the System

The FRA System is scoped for use within academic institutions such as universities and colleges. It covers all aspects of attendance management from user registration and dataset creation to live

face scanning, attendance locking, and report generation. The system operates as a web application accessible via modern web browsers on desktops and laptops equipped with webcams.

The scope includes three primary modules: Authentication & Role Management, Biometric Face Recognition Engine, and Academic & Report Management. The system supports the full lifecycle of an academic session including department creation, student enrollment, subject assignment, teacher verification, attendance session management, and export report.

1.1.3 Feasibility Analysis

- **Technical Feasibility**

The FRA System uses widely adopted and mature technologies. Python and Django provide a stable backend framework. TensorFlow and OpenCV are industry-standard tools for machine learning and computer vision. The frontend uses HTML5 with WebRTC for real-time webcam access, which is supported in all modern browsers. SQLite is used for development and can be easily migrated to PostgreSQL for production. All chosen technologies are open-source, well-documented, and technically feasible for the project scope.

- **Economic Feasibility**

All core technologies used in the FRA System are open-source and free to use, making the project economically viable with minimal software cost. The primary investment is in hardware (a server and webcam-equipped client machines). Cloud deployment options such as AWS or Digital Ocean can be used for scalable and cost-effective hosting. Compared to commercial biometric attendance hardware, this system offers a significantly more affordable alternative.

- **Operational Feasibility**

The system is designed to be intuitive and user-friendly. Teachers and students interact with the system through a clean web interface. Admin operations are centralized in a management dashboard. Minimal training is required. The replacement of paper-based and manual systems with this automated solution will be well received by institutional staff and students who seek faster, more accurate attendance management.

1.2 Stakeholders

1. University Administration: Oversees institutional policies, deploys the system, and monitors aggregate attendance data.
2. Department Administrator (Admin): Manages departments, sections, subjects, teachers, and students within the system.

3. Teachers: Conduct attendance sessions, verify their own identity via face scan, and access attendance records for their subjects.
4. Students: Participate in attendance sessions by scanning their faces and viewing their attendance records.
5. University Departments: Responsible for system installation, maintenance, and technical support.

1.3 Users of the System

The FRA System serves three categories of users, each with distinct roles and access privileges:

1. **Admin:** Full system access. Manage users, departments, sections, subjects, AI datasets, and system configurations. Reviews audit logs and generate system-wide reports.
2. **Teacher:** Can log in with password followed by face verification. Activates and manages attendance sessions. Views attendance reports for their assigned subjects. Has self-attendance capability.
3. **Student:** Registers and awaits admin approval. Participate in teacher-activated sessions. Scans face to mark attendance. Views personal attendance history.

1.4 Functional Requirements

1.4.1 List of Functional Requirements

1. User Registration with role-based assignment (Admin/Teacher/Student).
2. Admin approval workflow for new accounts before activation.
3. Teacher face verification gate after password-based login.
4. OTP verification for enhanced session security.
5. Face dataset creation for each registered student.
6. CNN model training and retraining on student face samples.
7. Live webcam face scan for student attendance marking.
8. Teacher-activated attendance sessions per subject.
9. Duplicate attendance prevention within a single session.
10. Biometric attendance record locking after session closure.
11. Department, Section, Subject, and Student enrollment management.
12. Daily, weekly, and monthly attendance report generation.
13. Audit log recording all actions with timestamps and IP addresses.
14. Real-time notifications and alerts for attendance events.
15. Camera and session management tools for admin.

1.4.2 Description of Each Requirement in Requirement Shell.

Requirement # 01		Requirement Type: Functional
Requirement: User Registration		
Description:	The system shall allow new users (Admin, Teacher, Student) to register an account specifying their role, personal details, and credentials. The account remains inactive until approved by an Admin.	
Rationale:	To control system access and ensure only authorized personnel can use the platform.	
Originator:	System Administrator / New Users	
Fit Criterion:	A registered user account exists in the database with status 'Pending'. Admin can see the request and activate it.	

Table 1 Functional requirement of user registration

Requirement # 02		Requirement Type: Functional
Requirement: Teacher Face Verification		
Description:	After successful password login, the system will require the teacher to complete a live face scan to verify identity before granting dashboard access.	
Rationale:	To prevent unauthorized access to teacher dashboards and prevent proxy logins.	
Originator:	Teacher	
Fit Criterion:	The teacher's face is matched against their stored biometric profile. If matched with sufficient cosine similarity, access is granted. Otherwise, access is denied.	

Table 2 Functional Requirement of Teacher Face Verification

Requirement # 03		Requirement Type: Functional
Requirement: Live Attendance Session		
Description:	A teacher shall be able to activate a live attendance session for a subject. Students in that subject will see the active session and scan their faces to mark attendance.	
Rationale:	To automate and secure the attendance marking process using biometric verification.	
Originator:	Teacher / Student	
Fit Criterion:	Students who complete successful face verification within the active session are marked present. Their record is locked after the session ends.	

Table 3 Functional Requirement of Live Attendance Session

Requirement # 04		Requirement Type: Functional
Requirement: Attendance Report Generation		
Description:	The system shall generate attendance reports broken down by day, week, and month for each subject, section, and department.	
Rationale:	To provide administrators and teachers with accurate and timely attendance data for academic monitoring and decision-making.	
Originator:	Admin / Teacher	
Fit Criterion:	Reports are generated and displayed accurately, matching the attendance records stored in the database.	

Table 4 Functional Requirement of Attendance Report Generation

Requirement # 05		Requirement Type: Functional
Requirement: Audit Log		
Description:	The system shall record every significant user action (login, attendance marking, session creation, user management) with a timestamp and the user's IP address.	
Rationale:	To maintain accountability, detect misuse, and support institutional auditing requirements.	
Originator:	Admin	
Fit Criterion:	All actions are recorded in the audit log table. Logs are viewable by admin and cannot be deleted by regular users.	

Table 5 Functional Requirement of Audit Log

1.5 Non-Functional Requirements

1. **Security:** All passwords are hashed. Sessions are secured with OTP and face biometrics. Attendance records are locked post-session to prevent tampering.
2. **Performance:** The face recognition engine shall process a live frame and return a recognition result within 3 seconds under normal conditions.
3. **Scalability:** The system architecture shall support scaling to multiple departments and hundreds of concurrent users without significant performance degradation.
4. **Usability:** The system shall provide a clean, intuitive web interface requiring no specialized technical knowledge to operate.
5. **Reliability:** The system shall maintain 99% uptime during active academic sessions and preserve all attendance data in the event of a partial system failure.
6. **Maintainability:** The codebase shall follow Django best practices with modular app structure, enabling future feature additions with minimal refactoring.
7. **Compatibility:** The system will run on standard modern web browsers (Chrome, Firefox, Edge) without requiring browser plugins or extensions.

1.6 Schedule of the Project

The project was developed following the Agile methodology with iterative sprints. Below is a summary of the development timeline:

Sprint	Activity	Duration
Sprint 1	Requirements gathering, system design, technology selection	Weeks 1–2
Sprint 2	Database design, user authentication module, role management	Weeks 3–4
Sprint 3	Face dataset creation, CNN model training pipeline	Weeks 5–6
Sprint 4	Live attendance session module, face scan integration	Weeks 7–8
Sprint 5	Reporting module, audit logs, notifications	Weeks 9–10
Sprint 6	Testing, bug fixing, UI polishing, deployment	Weeks 11–12

Table 6 Sprint Table

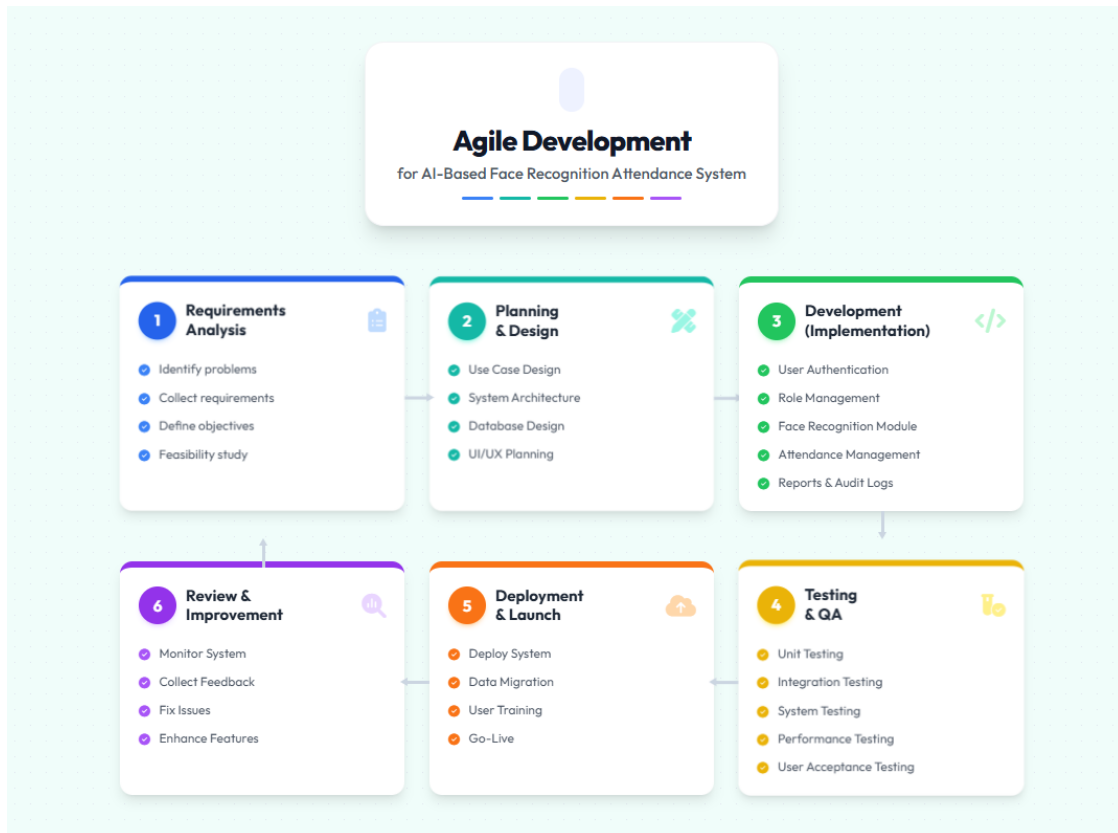


Figure 1 Agile Diagram

CHAPTER 2: Analysis

This chapter presents the analysis of the FRA System through use case models, system sequence diagrams, and a domain model. These artifacts capture the behavioral requirements of the system from the perspective of different actors and provide the foundation for the design phase.

2.1 Use Case Model

The FRA System involves three primary actors: Admin, Teacher, and Student. The use case model below identifies the key interactions each actor has with the system.

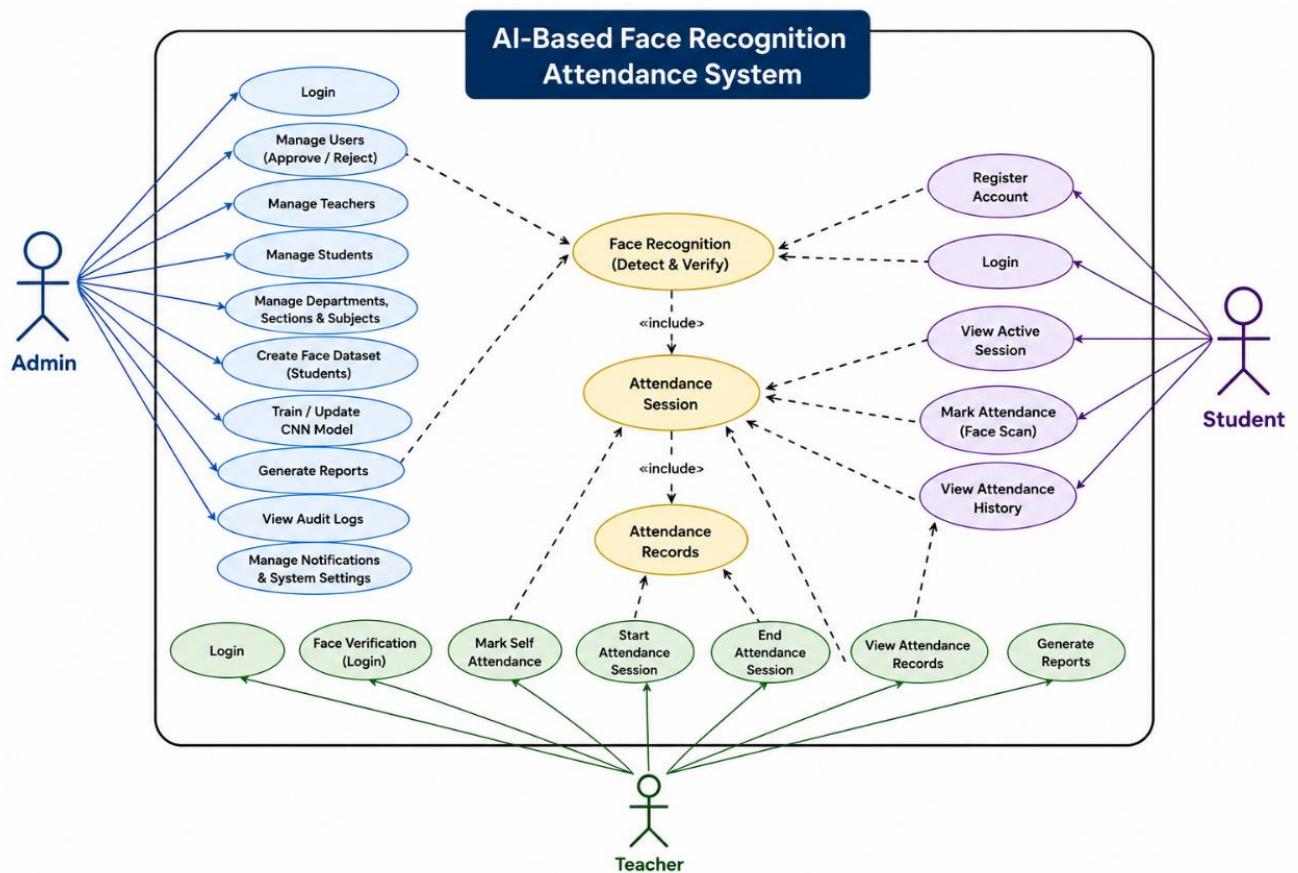


Figure 2 Use Case model

2.1.1 Description of Each Use Case

Use Case 1: User Registration

UC #	UC01
UC Name	User Registration
Level	Detailed
Description	A new user registers an account in the system by providing personal details, choosing a role (Student or Teacher), and setting credentials. The account remains inactive until approved by an Admin.
Actor(s)	New User (Student / Teacher)
Stakeholders	Admin, IT Department
Preconditions	System is accessible and registration page is available.
Success Guarantee	User account is created with 'Pending' status and Admin is notified.
Main Success Scenario	1. User navigates to registration page. 2. User fills in personal details and selects role. 3. User submits the registration form. 4. System validates input and creates account with 'Pending' status. 5. Admin receives notification of new registration request.
Extensions	If email already exists: system displays error and prompts re-entry.
Special Requirements	Internet connection required for system access
Frequency of Occurrence	Multiple times daily

Table 7 User Registration Use Case

Use Case 2: Teacher Face Verification Login

UC #	UC02
UC Name	Teacher Face Verification Login
Level	Detailed
Description	A teacher logs in with their password. Upon successful password verification, the system requires a live face scan to grant dashboard access.
Actor(s)	Teacher
Stakeholders	Admin, System Security Module
Preconditions	Teacher has an approved account with a registered face profile.
Success Guarantee	Teacher gains access to the dashboard after successful password and face verification.
Main Success Scenario	1. Teacher enters username and password. 2. System verifies credentials. 3. System prompts for live face scan. 4. Teacher positions face in front of webcam. 5. System captures frame and runs face recognition. 6. On successful match, system grants dashboard access.
Extensions	If face not recognized: system allows retry up to 3 times, then locks the session.
Special Requirements	Internet connection required for system access
Frequency of Occurrence	Multiple times daily

Table 8 Teacher Face Verification Login Use Case

Use Case 3: Live Attendance Session

UC #	UC03
UC Name	Live Attendance Session
Level	Detailed
Description	Teacher activates a live attendance session for a specific subject. Students scan their faces to mark attendance. The session is closed by the teacher, after which records are locked.
Actor(s)	Teacher, Student
Stakeholders	Admin, Biometric Engine
Preconditions	Teacher is verified and logged in. Students have approved accounts and registered face datasets.
Success Guarantee	Attendance records for all verified students are saved and locked.
Main Success Scenario	1. Teacher opens attendance session for a subject. 2. Students see the active session for their enrolled subject. 3. Student initiates face scan via webcam. 4. System runs face recognition and verifies identity. 5. On success, attendance is marked as present. 6. Teacher closes the session. 7. System locks all attendance records for that session.
Extensions	If face not recognized: student's attendance is not marked. If session is already closed: late submissions are rejected.
Special Requirements	Internet connection required for system access
Frequency of Occurrence	Multiple times daily

Table 9 Live Attendance Session Use Case

2.1.2 System Sequence Diagrams (SSD)

FR Attendance
Face Recognition Attendance System

System Online Biometric Secured

Create Your Account

Join the Face Recognition Attendance System — fill in your details and enroll your biometrics.

ACCOUNT INFORMATION

USERNAME*
[Input field]
Required. 150 characters or fewer. Letters, digits and @./+/-/_ only.

EMAIL*
[Input field]

FIRST NAME [Input field] LAST NAME [Input field]

STUDENT ID [Input field: STU-20260601-001] PHONE [Input field]

ROLE*
[Input field: Student]

PASSWORD* [Input field] [Eye icon]

PASSWORD CONFIRMATION* [Input field] [Eye icon]
Enter the same password as before, for verification.

DEPARTMENT [Dropdown: -- Select Department --] SECTION (REQUIRED FOR STUDENTS) [Dropdown: -- Select Department First --]

[Create Account]

BIOMETRIC ENROLLMENT

REQUIRED

Camera ready

[Capture] [Auto-Capture (5)]

0 / 5

Figure 3 Student Registration

FR Attendance
Face Recognition Attendance System

System Online | Biometric Secured

Create Your Account

Join the Face Recognition Attendance System — fill in your details and enroll your biometrics.

ACCOUNT INFORMATION

USERNAME*

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

EMAIL*

FIRST NAME | LAST NAME

EMPLOYEE ID | PHONE

EMP-20260601-001

ROLE*

Teacher

PASSWORD*

PASSWORD CONFIRMATION*

Enter the same password as before, for verification.

DEPARTMENT | SECTION (NOT REQUIRED FOR TEACHERS)

-- Select Department -- | -- Select Department First --

Create Account

BIOMETRIC ENROLLMENT REQUIRED

Camera ready

Capture | Auto-Capture (5)

0/5

Figure 4 Teacher Registration

Identity Verification

Please verify your identity to access the portal

Muhammad Ehtisham | Teacher

Camera preview will appear here

Verify My Identity

Cancel & Logout

Figure 5 Teacher Login Verification

My Subjects				
Subject Code	Subject Name	Department	Section	Actions
ICT-334	ICT	ICT	Section-Morning	View **Live Attendance
MATH-76	MATH	ICT	Section-Morning	View **Live Attendance

Figure 6 Live Attendance Session

Mark Attendance

Choose your preferred method to mark attendance

Face Recognition

Retina Recognition

COMING SOON

Figure 7 Student Marking Attendance using face recognition

Mark Attendance — Face Recognition

Your face is verified against your registered samples only

Select Subject
Live

Start Scan

[← Back to Dashboard](#)

Figure 8 Student Attendance Mark Using Face Recognition

2.1.3 Domain Model

The domain model identifies the key entities in the FRA System and their relationships. The primary entities include:

User, Admin, Teacher, Student, Department, Section, Subject, Attendance Session, Attendance Record, Face Dataset, CNN Model, Audit Log, and Notification.

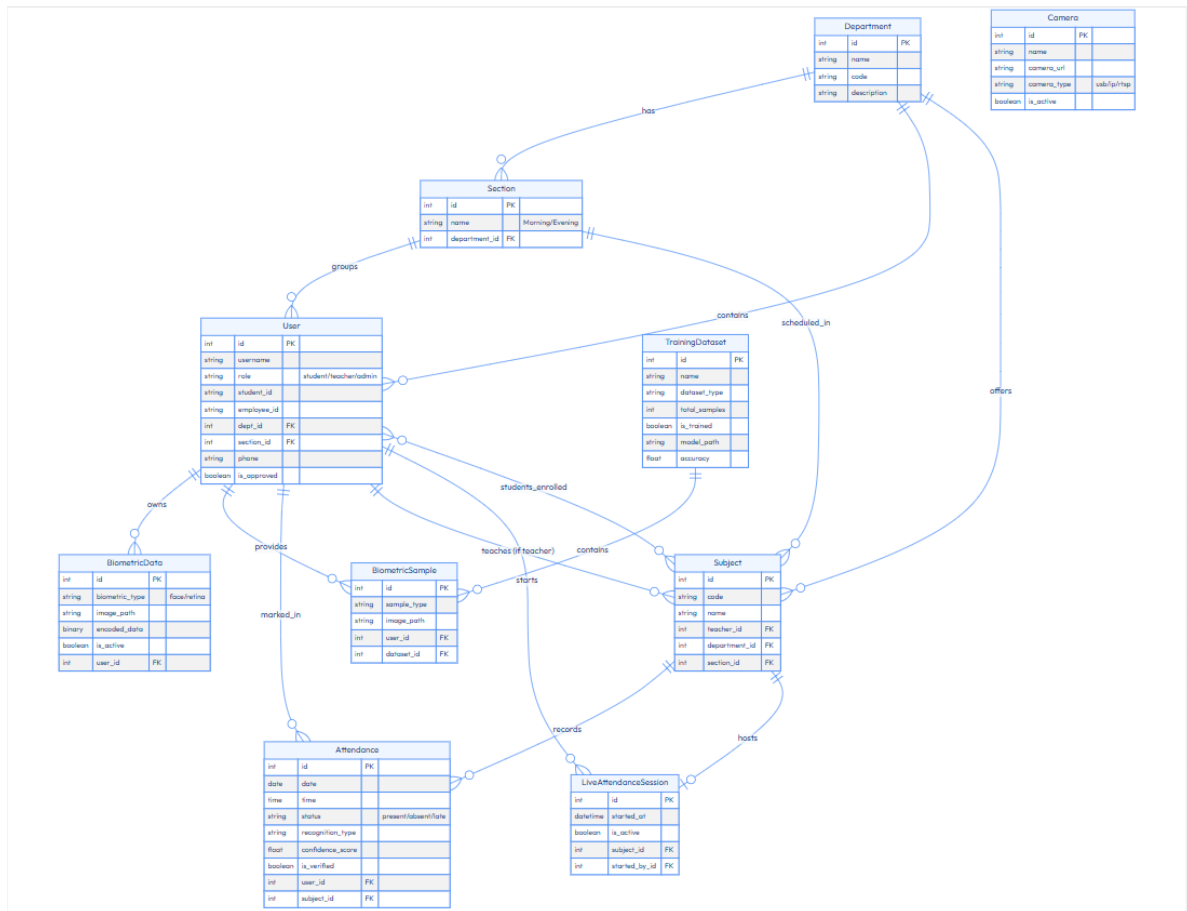


Figure 8 Domain Model

CHAPTER 3: Project Design

This chapter presents the complete design of the FRA System including class diagrams, entity relationship diagrams, sequence diagrams, and system architecture.

3.1 Design Class Diagram

The design class diagram extends the domain model by adding attributes, methods, and associations with multiplicity. Key classes include User (with subclasses Admin, Teacher, Student), Attendance Session, Attendance Record, Face Dataset, CNN Model Manager, Audit Log, Department, Section, and Subject.

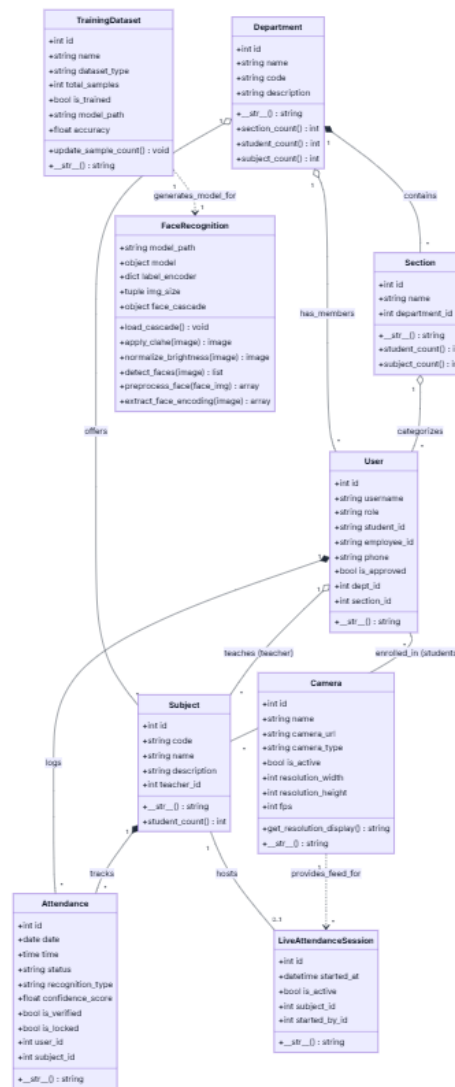


Figure 9 Design Class Diagram

3.2 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram below represents the database schema for the FRA System. The central entities are User, Teacher, Student, Department, Section, Subject, Enrollment, Attendance Session, Attendance Record, Face Dataset, Audit Log, and Notification.

Key relationships include:

1. A Department has many Sections. A Section belongs to one Department.
2. A Section has many Subjects. A Subject belongs to one Section.
3. A Student can be enrolled in multiple Subjects. An Enrollment links a Student to a Subject.
4. A Teacher is assigned to one or more Subjects.
5. An Attendance Session is created by a Teacher for a Subject.
6. An Attendance Record belongs to an Attendance Session and a Student.
7. A Face Dataset belongs to a Student and is used by the CNN Model for training and recognition.

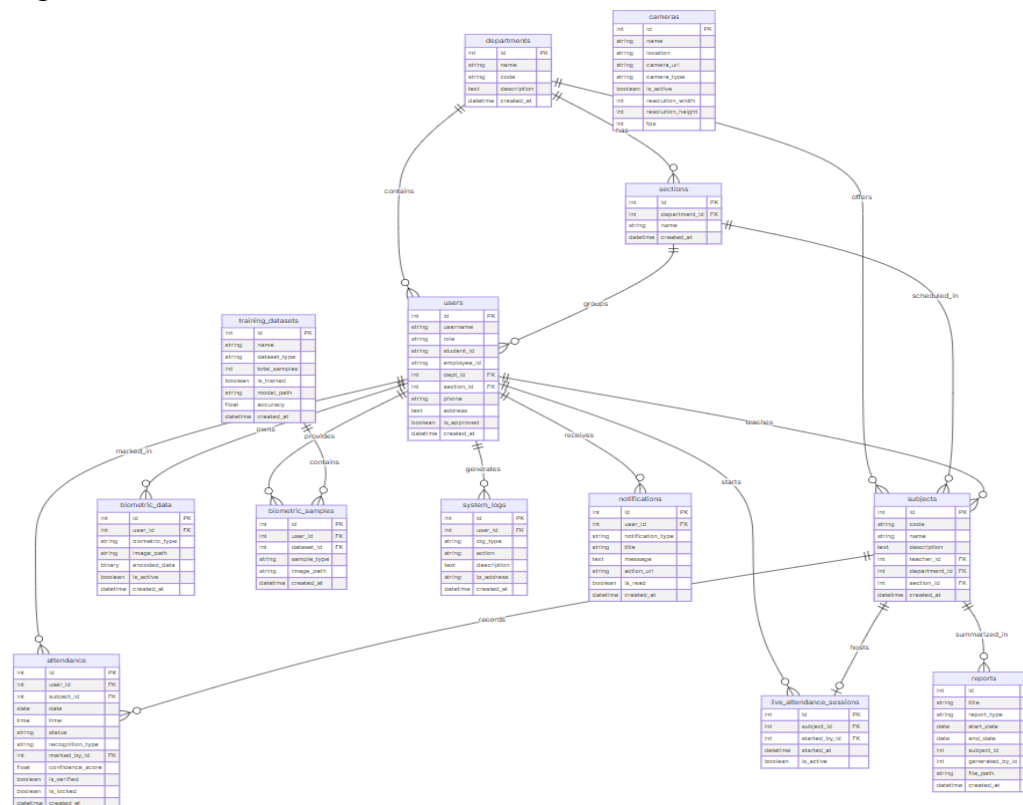


Figure 10 Entity Relationship Model

3.3 Sequence Diagrams

Sequence diagrams describe the dynamic interactions between objects for the main use cases.

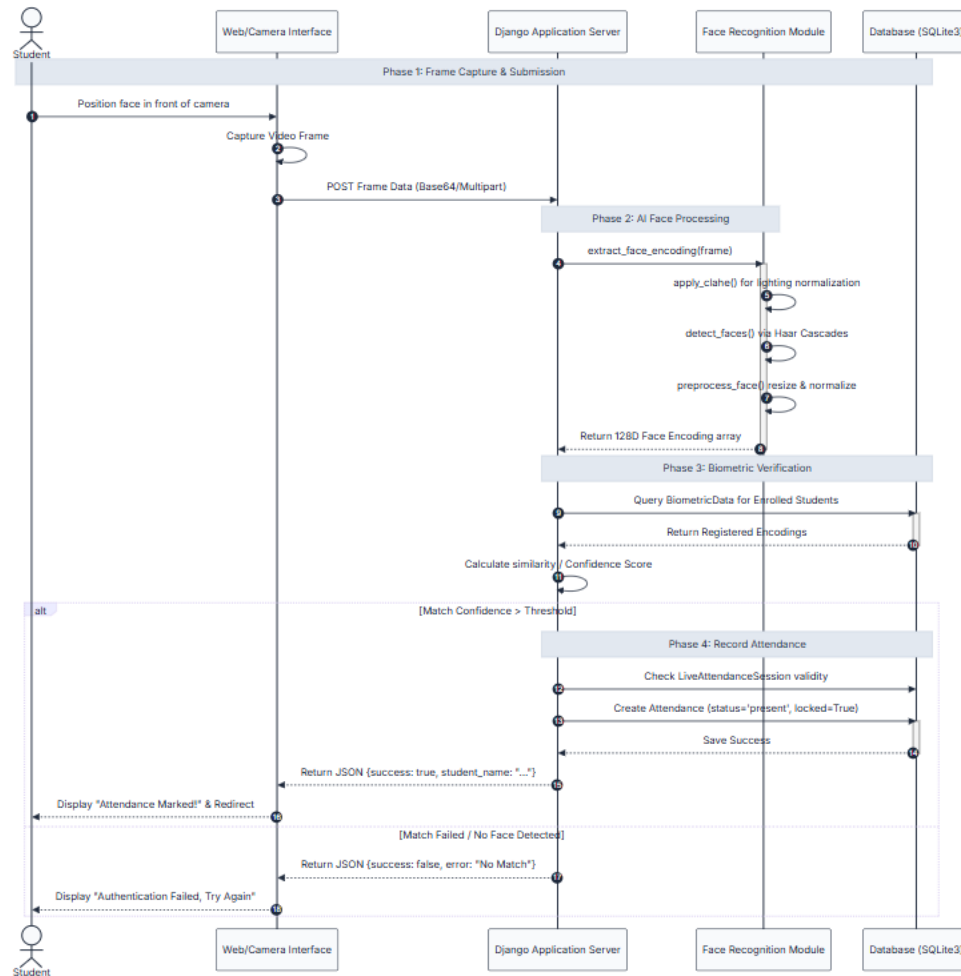


Figure 11 Sequence Diagram

3.4 System Architecture

The FRA System follows a 3-Tier Architecture consisting of the Presentation Layer, Application/Business Logic Layer, and Data Layer.

1. Presentation Layer: HTML5, CSS3, Bootstrap, JavaScript, WebRTC for live webcam access and Fetch API for asynchronous requests.
2. Application Layer: Python/Django handles routing, business logic, session management, and API endpoints. The AI pipeline (TensorFlow, OpenCV, scikit-learn) processes face recognition requests.
3. Data Layer: SQLite database stores all user, attendance, audit, and academic data.

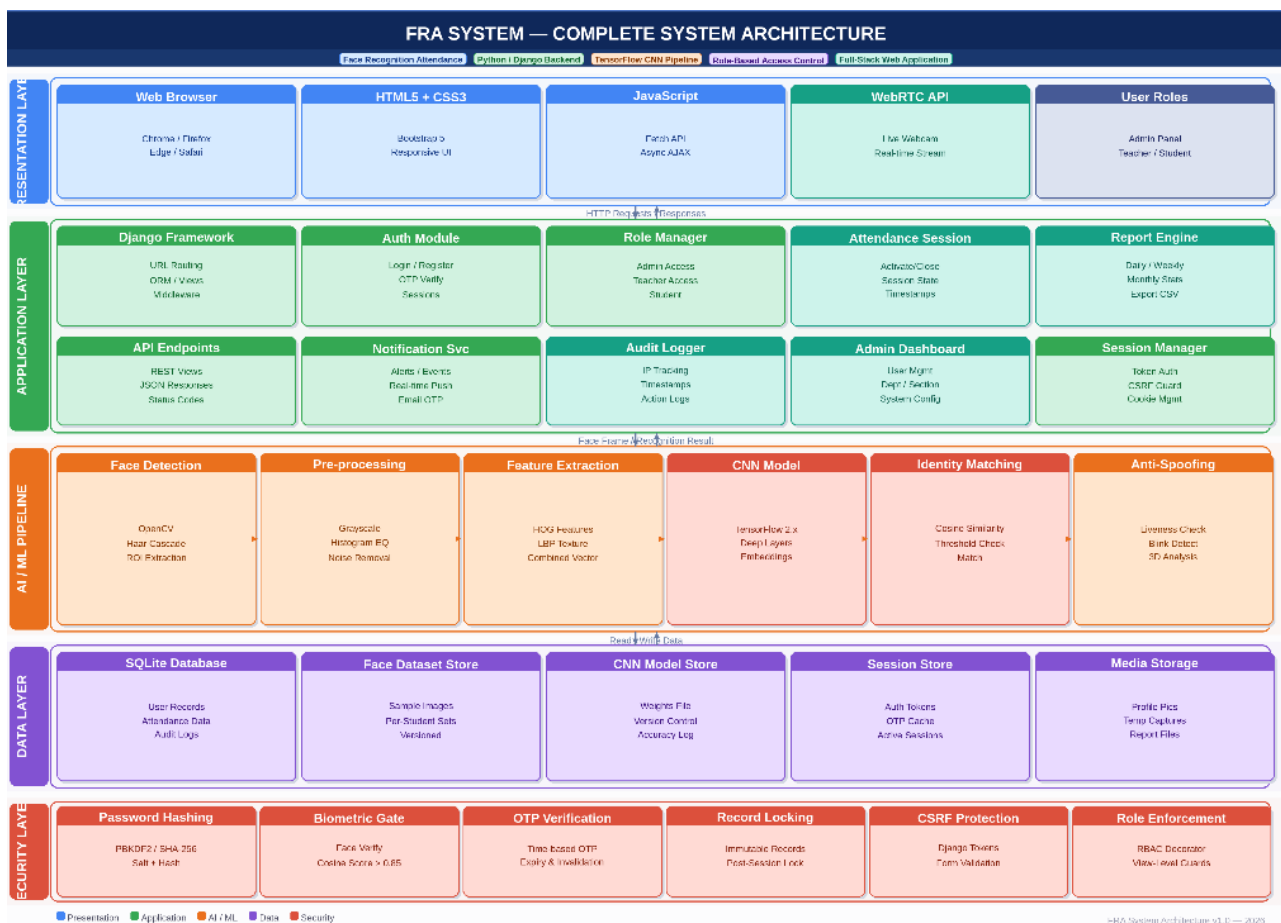


Figure 12 System Architecture

CHAPTER 4: Implementation

4.1 Tools & Technologies Used

Category	Technology / Tool	Purpose
Backend	Python 3.10 / Django 4.x	Web framework, business logic, routing
AI / ML	TensorFlow 2.x	CNN model training and inference
Computer Vision	OpenCV 4.x	Face detection (Haar Cascade), image processing
Feature Extraction	HOG + LBP (scikit-learn)	Texture and gradient-based face features
Numerical Computing	NumPy	Matrix operations for face vectors
Frontend	HTML5, CSS3, Bootstrap 5	UI design and responsive layout
Frontend JS	JavaScript, Fetch API	Async requests and dynamic UI
Webcam	WebRTC	Real-time video stream in browser
Database	SQLite	Data persistence (dev environment)
IDE	Visual Studio Code	Development environment
Version Control	Git / GitHub	Source code management

4.2 System Modules Description

- **Module 1: Authentication & Role Management**

This module handles the complete user lifecycle from registration to login and access control. When a new user registers, their account is set to 'Pending' status. The Admin reviews and approves or rejects registrations. Once approved, Teachers must complete a face verification step after password login. OTP verification is also available for an additional security layer. Session tokens are managed securely with Django's built-in session framework.

- **Module 2: Biometric Face Recognition Engine**

This is the core AI module of the FRA System. It consists of three stages: face detection, feature extraction, and identity matching. Face detection uses OpenCV's Haar Cascade Classifier to locate face regions in webcam frames. Feature extraction combines HOG (Histogram of Oriented Gradients) and LBP (Local Binary Patterns) descriptors to generate a comprehensive feature

vector. A CNN model trained on student face samples performs deep learning-based identity recognition. Final identity matching uses cosine similarity to compare the live face embedding with stored profiles.

- **Module 3: AI Dataset & Model Management**

Administrators can initiate face dataset collection sessions for students, capturing multiple face samples under different angles and lighting conditions. These samples are stored and used to train or retrain the CNN model. The admin dashboard provides controls to initiate training jobs, monitor training progress, and update the deployed model.

- **Module 4: Attendance Session Management**

Teachers activate attendance sessions for their assigned subjects. Students enrolled in the subject can see the active session and initiate a face scan. The system verifies the student's identity and marks them present. The session can be closed by the teacher, after which all records are locked and cannot be modified. Duplicate attendance within the same session is automatically prevented.

- **Module 5: Reports, Audit & Notifications**

The system generates detailed attendance reports at various time granularities (daily, weekly, monthly) broken down by subject, section, and department. Every system action is recorded in the audit log with the actor, action type, timestamp, and IP address. Real-time notifications alert users to relevant events such as session activation or absence thresholds.

- **Module 6: Teacher Face Verification Gate (Login Security)**

This module provides an extra security layer where teachers must pass live face verification after password login. The system uses HOG (Histogram of Oriented Gradients) + LBP (Local Binary Pattern) matching to confirm identity before dashboard access is granted.

- **Module 7: Teacher Self-Attendance (Mark My Attendance)**

This module allows teachers to mark their own attendance through a live webcam face scan. The system uses HOG (Histogram of Oriented Gradients) + LBP (Local Binary Pattern) verification and prevents duplicate attendance records for the same day.

- **Module 8: Live Attendance Session (Student Attendance)**

This module allows teachers to start a live attendance session for a specific subject and time window. Students can mark attendance only during the active session using AI-based face

verification with HOG (Histogram of Oriented Gradients), LBP (Local Binary Pattern), and CNN (Convolutional Neural Network) recognition.

- **Module 9: Academic Structure Management**

This module manages the academic hierarchy including Department → Section → Subject → Students. It helps admins and teachers organize student enrollment, subject allocation, and section-based attendance management efficiently.

- **Module 10: Camera & Notification Management**

This module manages USB/IP/RTSP camera settings for live face capture and attendance monitoring. It also provides real-time notifications for attendance updates, session activity, approvals, and system alerts.

4.3 Code / Algorithm of Main Scenarios

4.3.1 Face Recognition.py

```
import cv2
import numpy as np
import os
from pathlib import Path
import pickle

from tensorflow import keras
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.utils import to_categorical

class FaceRecognition:
    _clahe_detect = None # CLAHE for detection (clipLimit=3.0, grid=8x8)
    _clahe_patch = None # CLAHE for patch norm (clipLimit=3.0, grid=4x4)
    _hog64 = None # HOGDescriptor for 64×64 full-face
    _hog32 = None # HOGDescriptor for 32×32 zone patches

    @classmethod
    def _get_clahe_detect(cls):
        if cls._clahe_detect is None:
            cls._clahe_detect = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
        return cls._clahe_detect

    @classmethod
    def _get_clahe_patch(cls):
        if cls._clahe_patch is None:
```

```

        cls._clahe_patch = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(4, 4))
    return cls._clahe_patch

    @classmethod
    def _get_hog64(cls):
        if cls._hog64 is None:
            cls._hog64 = cv2.HOGDescriptor(
                _winSize=(64, 64), _blockSize=(16, 16),
                _blockStride=(8, 8), _cellSize=(8, 8), _nbins=9,
            )
        return cls._hog64

    @classmethod
    def _get_hog32(cls):
        if cls._hog32 is None:
            cls._hog32 = cv2.HOGDescriptor(
                _winSize=(32, 32), _blockSize=(16, 16),
                _blockStride=(8, 8), _cellSize=(8, 8), _nbins=9,
            )
        return cls._hog32

    def __init__(self, model_path=None):
        self.model_path = model_path
        self.model = None
        self.label_encoder = {} # Maps user_id to class index
        self.face_cascade = None
        self.img_size = (128, 128)
        self.load_cascade()

        # Automatically load the latest model if no path specified
        if model_path is None:
            self.auto_load_latest_model()
        elif os.path.exists(model_path):
            self.load_model(model_path)

    def load_cascade(self):
        """Load Haar Cascade for face detection"""
        try:
            cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
            self.face_cascade = cv2.CascadeClassifier(cascade_path)
        except Exception as e:
            print(f"Error loading cascade: {e}")

```

```

def apply_clahe(self, image):
    """
    Apply CLAHE (Contrast Limited Adaptive Histogram Equalization) to improve
    face detection under poor / uneven lighting conditions.
    Works on BGR images; returns BGR.
    Uses the cached CLAHE object to avoid repeated construction overhead.
    """
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l_channel, a, b = cv2.split(lab)
    clahe = self._get_clahe_detect()
    l_channel = clahe.apply(l_channel)
    lab = cv2.merge((l_channel, a, b))
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

def normalize_brightness(self, image):
    """
    Gamma correction to brighten very dark frames captured by low-quality webcams.
    """
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    mean_brightness = np.mean(gray)
    if mean_brightness < 80:
        # Dark frame — boost brightness
        gamma = 0.5
        inv_gamma = 1.0 / gamma
        table = np.array([((i / 255.0) ** inv_gamma) * 255
                          for i in np.arange(0, 256)]).astype("uint8")
        return cv2.LUT(image, table)
    return image

# Face detection

def detect_faces(self, image):
    """
    Detect faces — fast two-pass approach.
    Pass 1 (strict): scaleFactor=1.1, minNeighbors=4, minSize=(40,40)
    Pass 2 (relaxed): scaleFactor=1.05, minNeighbors=2, minSize=(30,30)
    Uses cached CLAHE; tries enhanced then raw gray as fallback.
    Returns a list of (x, y, w, h) bounding boxes.
    """
    if self.face_cascade is None:
        return []

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Use cached CLAHE — avoids repeated object creation
    gray_enhanced = self._get_clahe_detect().apply(gray)

```

```

# Fast two-pass: strict first (usually succeeds), relaxed as fallback
detection_configs = [
    (gray_enhanced, 1.1, 4, (40, 40)), # Pass 1 – strict + enhanced
    (gray_enhanced, 1.05, 2, (30, 30)), # Pass 2 – relaxed + enhanced
    (gray, 1.1, 3, (40, 40)), # Pass 3 – raw fallback
]
for detect_gray, scale, neighbors, min_size in detection_configs:
    faces = self.face_cascade.detectMultiScale(
        detect_gray,
        scaleFactor=scale,
        minNeighbors=neighbors,
        minSize=min_size,
        flags=cv2.CASCADE_SCALE_IMAGE,
    )
    if len(faces) > 0:
        return faces

return []

def preprocess_face(self, face_img):
    """Preprocess face image for recognition"""
    face_resized = cv2.resize(face_img, self.img_size)
    face_normalized = face_resized.astype('float32') / 255.0
    return face_normalized

# Encoding extraction (used for Pearson-correlation matching)

def extract_face_encoding(self, image):
    """
    Extract face encoding from image.

    Tries, in order:
    1. Raw image.
    2. CLAHE-enhanced image.
    3. Brightness-normalised image.
    4. Horizontally flipped image (webcam mirror mismatch).

    Returns a float32 (128×128×3) numpy array, or None if no face found.

    candidates = [image]

```

4.3.2 Teacher Attendance Session:

{% block title %}Teacher Attendance - FRA System{% endblock %}

{% block content %}

```
<div class="container mt-4">
  <div class="row">
    <div class="col-12">
      <div class="d-flex justify-content-between align-items-center mb-4">
        <h2 class="gradient-text">
          <i class="fas fa-chalkboard-teacher"></i> Teacher Attendance Management
        </h2>
        <a href="{% url 'attendance_history' %}?role=teacher" class="btn btn-outline-
primary">
          <i class="fas fa-history"></i> View Teacher History
        </a>
      </div>
    </div>

    <!-- Quick Stats -->
    <div class="row mb-4">
      <div class="col-md-3">
        <div class="glass-card p-3 text-center">
          <i class="fas fa-calendar-day fa-2x mb-2 text-primary"></i>
          <h4>{{ today_attendance_count }}</h4>
          <small class="text-muted">Today's Attendance</small>
        </div>
      </div>
      <div class="col-md-3">
        <div class="glass-card p-3 text-center">
          <i class="fas fa-chalkboard-teacher fa-2x mb-2 text-success"></i>
          <h4>{{ total_teachers }}</h4>
          <small class="text-muted">Total Teachers</small>
        </div>
      </div>
      <div class="col-md-3">
        <div class="glass-card p-3 text-center">
          <i class="fas fa-check-circle fa-2x mb-2 text-info"></i>
          <h4>{{ present_count }}</h4>
          <small class="text-muted">Present Today</small>
        </div>
      </div>
      <div class="col-md-3">
        <div class="glass-card p-3 text-center">
          <i class="fas fa-times-circle fa-2x mb-2 text-danger"></i>
          <h4>{{ absent_count }}</h4>
          <small class="text-muted">Absent Today</small>
        </div>
      </div>
    </div>
  </div>
```

```

    </div>
</div>

<!-- Tabs -->
<ul class="nav nav-tabs mb-4" id="teacherAttendanceTabs" role="tablist">
    <li class="nav-item" role="presentation">
        <button class="nav-link active" id="ta-mark-tab" data-bs-toggle="tab" data-bs-
target="#ta-mark"
            type="button">
            <i class="fas fa-check"></i> Mark Attendance
        </button>
    </li>
    <li class="nav-item" role="presentation">
        <button class="nav-link" id="ta-edit-tab" data-bs-toggle="tab" data-bs-target="#ta-
edit"
            type="button">
            <i class="fas fa-edit"></i> Edit Attendance
        </button>
    </li>
    <li class="nav-item" role="presentation">
        <button class="nav-link" id="ta-bulk-tab" data-bs-toggle="tab" data-bs-target="#ta-
bulk"
            type="button">
            <i class="fas fa-list"></i> Bulk Operations
        </button>
    </li>
</ul>

<!-- Tab Content -->
<div class="tab-content" id="teacherAttendanceTabContent">

    <!-- Mark Attendance Tab -->
    <div class="tab-pane fade show active" id="ta-mark" role="tabpanel">
        <div class="glass-card p-4">
            <h4 class="mb-4">Mark Individual Teacher Attendance</h4>

            {% if not subjects %}
            <div class="alert alert-warning">
                <i class="fas fa-exclamation-triangle"></i>
                No subjects found. Please create subjects first before marking attendance.
            </div>
            {% else %}
            <form method="POST" action="{% url 'teacher_mark_attendance' %}">
                {% csrf_token %}
                <div class="row">
                    <div class="col-md-6 mb-3">

```

```

        <label class="form-label">Teacher</label>
        <select class="form-select" name="user_id" required>
            <option value="">Select teacher...</option>
            {% for teacher in teachers %}
            <option value="{{ teacher.id }}">{{
teacher.get_full_name|default:teacher.username }}
            {% if teacher.employee_id %}({{ teacher.employee_id }}){% endif
            %}</option>
            {% endfor %}
        </select>
    </div>
    <div class="col-md-6 mb-3">
        <label class="form-label">Subject / Session</label>
        <select class="form-select" name="subject_id" required>
            <option value="">Select subject...</option>
            {% for subject in subjects %}
            <option value="{{ subject.id }}">{{ subject.name }} ({{ subject.code
}}})</option>
            {% endfor %}
        </select>
    </div>
</div>

<div class="row">
    <div class="col-md-4 mb-3">
        <label class="form-label">Date</label>
        <input type="date" class="form-control" name="date" value="{{ today
}}} " required>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Status</label>
        <select class="form-select" name="status" required>
            <option value="present">Present</option>
            <option value="absent">Absent</option>
            <option value="late">Late</option>
        </select>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Time</label>
        <input type="time" class="form-control" name="time" value="{{
current_time|time:'H:i' }}">
    </div>
</div>

<div class="mb-3">
    <label class="form-label">Notes (Optional)</label>

```

```

        <textarea class="form-control" name="notes" rows="3"
            placeholder="Add any notes or reasons..."></textarea>
    </div>

    <button type="submit" class="btn btn-primary">
        <i class="fas fa-save"></i> Mark Attendance
    </button>
</form>
{% endif %}
</div>
</div>

```

<!-- Edit Attendance Tab -->

```

<div class="tab-pane fade" id="ta-edit" role="tabpanel">
    <div class="glass-card p-4">
        <h4 class="mb-4">Edit Teacher Attendance Records</h4>

        <!-- Filter Form -->
        <form method="GET" class="mb-4">
            <div class="row">
                <div class="col-md-3">
                    <input type="date" class="form-control" name="filter_date"
                        value="{{ filter_date }}" placeholder="Date">
                </div>
                <div class="col-md-3">
                    <select class="form-select" name="filter_subject">
                        <option value="">All Subjects</option>
                        {% for subject in subjects %}
                        <option value="{{ subject.id }}"
                            {% if filter_subject == subject.id|slugify %}selected{% endif %}>
                            {{ subject.name }} ({{ subject.code }})
                        </option>
                        {% endfor %}
                    </select>
                </div>
                <div class="col-md-3">
                    <select class="form-select" name="filter_status">
                        <option value="">All Status</option>
                        <option value="present" {% if filter_status == 'present' %}selected{%
endif %}>Present</option>
                        <option value="absent" {% if filter_status == 'absent' %}selected{%
endif %}>Absent</option>
                        <option value="late" {% if filter_status == 'late' %}selected{% endif
%}>Late</option>
                    </select>
                </div>
            </div>
        </form>
    </div>
</div>

```


4.3.3 admin_panel.html:

```
{% extends 'base.html' %}

{% block title %}Admin Panel - FRA System{% endblock %}

{% block content %}
<div class="container py-5">
  <div class="row mb-4">
    <div class="col-12">
      <h1 class="gradient-text mb-3">
        <i class="fas fa-user-shield"></i> Admin Control Panel
      </h1>
      <p class="text-muted">Manage users, approvals, and system settings</p>
    </div>
  </div>

  <!-- Statistics Cards -->
  <div class="row mb-4">
    <div class="col-md-4 mb-3">
      <div class="stat-card">
        <div class="stat-icon bg-primary">
          <i class="fas fa-users"></i>
        </div>
        <div class="stat-content">
          <h3>{{ all_users.count }}</h3>
          <p>Total Users</p>
        </div>
      </div>
    </div>
    <div class="col-md-4 mb-3">
      <div class="stat-card">
        <div class="stat-icon bg-warning">
          <i class="fas fa-clock"></i>
        </div>
        <div class="stat-content">
          <h3>{{ pending_users.count }}</h3>
          <p>Pending Approvals</p>
        </div>
      </div>
    </div>
    <div class="col-md-4 mb-3">
      <div class="stat-card">
        <div class="stat-icon bg-success">
          <i class="fas fa-check-circle"></i>
        </div>
        <div class="stat-content">

```

```

    <h3>{{ all_users|length|add:pending_users.count|add:"-"|add:pending_users.count
}}</h3>
    <p>Active Users</p>
  </div>
</div>
</div>
</div>
</div>

<!-- Pending Approvals -->
{% if pending_users %}
<div class="card glass-card mb-4">
  <div class="card-header">
    <h4 class="mb-0">
      <i class="fas fa-user-clock"></i> Pending User Approvals
      <span class="badge bg-warning">{{ pending_users.count }}</span>
    </h4>
  </div>
  <div class="card-body">
    <div class="table-responsive">
      <table class="table table-hover">
        <thead>
          <tr>
            <th>Username</th>
            <th>Email</th>
            <th>Full Name</th>
            <th>Role</th>
            <th>Dept / Section</th>
            <th>Registered</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {% for user in pending_users %}
          <tr>
            <td><strong>{{ user.username }}</strong></td>
            <td>{{ user.email }}</td>
            <td>{{ user.first_name }} {{ user.last_name }}</td>
            <td>
              <span class="badge bg-info">{{ user.role|upper }}</span>
            </td>
            <td>
              {% if user.dept %}
              <span class="badge rounded-pill px-2 py-1 d-block mb-1"
style="background:rgba(102,126,234,.2);color:#a5b4fc;border:1px solid
rgba(102,126,234,.3);font-size:.68rem;">
                <i class="fas fa-university me-1"></i>{{ user.dept.code }}

```

```

        </span>
        {% endif %}
        {% if user.section %}
        <span class="badge rounded-pill px-2 py-1 d-block"
style="background:rgba(17,153,142,.2);color:#38ef7d;border:1px solid rgba(17,153,142,.3);font-
size:.68rem;">
            <i class="fas fa-layer-group me-1"></i>{{ user.section.name }}
        </span>
        {% else %}
        <small class="text-muted">—</small>
        {% endif %}
    </td>
    <td>{{ user.date_joined|date:"M d, Y" }}</td>
    <td>
        <a href="{% url 'approve_user' user.id %}" class="btn btn-sm btn-success
me-1"
            onclick="return confirm('Approve user {{ user.username }}?')">
            <i class="fas fa-check"></i> Approve
        </a>
        <a href="{% url 'delete_user' user.id %}" class="btn btn-sm btn-danger"
            onclick="return confirm('Are you sure you want to delete user {{
user.username }}? This action cannot be undone!')">
            <i class="fas fa-trash"></i> Delete
        </a>
    </td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>
</div>
{% else %}
<div class="alert alert-info">
    <i class="fas fa-info-circle"></i> No pending user approvals at this time.
</div>
{% endif %}

<!-- All Users -->
<div class="card glass-card mb-4">
    <div class="card-header">
        <h4 class="mb-0">
            <i class="fas fa-users"></i> All Users
        </h4>
    </div>
    <div class="card-body">

```

```

<div class="table-responsive">
  <table class="table table-hover">
    <thead>
      <tr>
        <th>ID</th>
        <th>Photo</th>
        <th>Username</th>
        <th>Email</th>
        <th>Role</th>
        <th>Dept / Section</th>
        <th>Status</th>
        <th>Last Login</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {% for user in all_users %}
      <tr>
        <td>{{ user.id }}</td>
        <td>
          {% if user.face_photo_path %}
          
          {% else %}
          <span style="display:inline-flex;align-items:center;justify-
content:center;width:38px;height:38px;border-
radius:50%;background:#e9ecef;color:#6c757d;font-size:1.1rem;">
            <i class="fas fa-user"></i>
          </span>
          {% endif %}
        </td>
        <td><strong>{{ user.username }}</strong></td>
        <td>{{ user.email }}</td>
        <td>
          <span
            class="badge bg-{% if user.role == 'admin' %}danger{% elif user.role ==
'teacher' %}primary{% else %}secondary{% endif %}">
            {{ user.role|upper }}
          </span>
        </td>
        <td>
          {% if user.dept %}

```

```

        <span class="badge rounded-pill px-2 py-1 d-block mb-1"
style="background:rgba(102,126,234,.2);color:#a5b4fc;border:1px solid
rgba(102,126,234,.3);font-size:.65rem;white-space:nowrap;">
        <i class="fas fa-university me-1"></i>{{ user.dept.code }}
    </span>
    {% endif %}
    {% if user.section %}
    <span class="badge rounded-pill px-2 py-1 d-block"
style="background:rgba(17,153,142,.2);color:#38ef7d;border:1px solid rgba(17,153,142,.3);font-
size:.65rem;white-space:nowrap;">
        <i class="fas fa-layer-group me-1"></i>{{ user.section.name }}
    </span>
    {% else %}
    <small class="text-muted">—</small>
    {% endif %}
</td>
<td>
    {% if user.is_approved %}
    <span class="badge bg-success">Approved</span>
    {% else %}
    <span class="badge bg-warning">Pending</span>
    {% endif %}
</td>
<td>
    {% if user.last_login %}
    {{ user.last_login|date:"M d, Y H:i" }}
    {% else %}
    <span class="text-muted">Never</span>
    {% endif %}
</td>
<td>
    {% if user.id != request.user.id %}
    <a href="{% url 'delete_user' user.id %}" class="btn btn-sm btn-danger"
onclick="return confirm('Are you sure you want to delete user {{
user.username }}? This action cannot be undone!')">
        <i class="fas fa-trash"></i> Delete
    </a>
    {% else %}
    <span class="text-muted small">Current User</span>
    {% endif %}
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>

```

CHAPTER 5: System Testing

This chapter presents the test cases developed for the FRA System. Testing was performed across multiple levels including unit testing, integration testing, system testing, and user acceptance testing (UAT).

5.1 Test Cases

Test Case #:	TC-01	Test Case Name:	User Registration
System:	FRA System	Sub System:	Authentication Module
Designed By:	Development Team		
Short Description:	Verify that a new user can register an account with valid details and the account is created with Pending status.		
Steps:	1. Navigate to registration page. 2. Enter valid name, email, password, and role. 3. Submit form.		
Expected Result:	Account created with status 'Pending'. Success message displayed. Admin notified.		
Status:	Pass		

Table 10 User Registration Test Case

Test Case #:	TC-02	Test Case Name:	Teacher Face Verification Login
System:	FRA System	Sub System:	Biometric Module
Designed By:	Development Team		
Short Description:	Verify that a teacher cannot access the dashboard without completing face verification after password login.		
Steps:	1. Enter valid teacher credentials. 2. Skip face scan and try to access dashboard directly.		
Expected Result:	System redirects back to face scan page. Dashboard access denied without biometric verification.		
Status:	Pass		

Table 11 Teacher Face Recognition Login Test Case

Test Case #:	TC-03	Test Case Name:	Student Live Attendance Marking
System:	FRA System	Sub System:	Attendance Module
Designed By:	Development Team		
Short Description:	Verify that a student can mark attendance via face scan during an active session.		
Steps:	1. Teacher activates session for subject. 2. Student logs in and sees active session. 3. Student initiates face scan. 4. Camera captures face and system processes recognition.		
Expected Result:	Student's attendance is marked 'Present'. Record appears in session log. Duplicate marking is prevented.		
Status:	Pass		

Table 12 Student Live Attendance Test Case

Test Case #:	TC-04	Test Case Name:	Attendance Record Locking
System:	FRA System	Sub System:	Attendance Module
Designed By:	Development Team		
Short Description:	Verify that attendance records are locked after the teacher closes the session.		
Steps:	1. Teacher closes an active attendance session. 2. Attempt to modify an attendance record for that session.		
Expected Result:	System prevents any modification to records of a closed session. All records remain locked.		
Status:	Pass		

Table 13 Attendance Record Locking Test Case

Test Case #:	TC-05	Test Case Name:	Report Generation
System:	FRA System	Sub System:	Reporting Module
Designed By:	Development Team		
Short Description:	Verify that the system generates accurate attendance reports filtered by date range, subject, and section.		
Steps:	1. Admin navigates to Reports section. 2. Selects subject, section, and date range. 3. Clicks Generate Report.		
Expected Result:	Accurate attendance report is displayed matching database records. Report can be exported.		
Status:	Pass		

Table 14 Report Generation Test Case

5.2 Testing Types

- **Unit Testing**

Each individual function and module was tested in isolation. The face recognition pipeline components (Haar Cascade detection, HOG/LBP feature extraction, cosine similarity matching) were tested with predefined face samples to verify correctness of output.

- **Integration Testing**

Modules were integrated progressively. The authentication module was tested with the biometric module to verify the teacher face verification gate. The attendance session module was tested with the face recognition engine to validate the complete attendance workflow.

- **System Testing**

The complete system was tested end-to-end simulating real academic scenarios: multiple students marking attendance in a live session, report generation, audit logging, and admin management operations.

- **User Acceptance Testing (UAT)**

A group of representative users (admin, teachers, and students) was selected to test the system in a controlled environment. Feedback was collected and critical usability issues were addressed before final deployment.

CHAPTER 6: User Interfaces

This chapter presents screenshots of the working FRA System user interfaces. The system features a clean, modern web interface built with Bootstrap 5, providing responsive and intuitive user experiences for all three user roles.

6.1 Interface

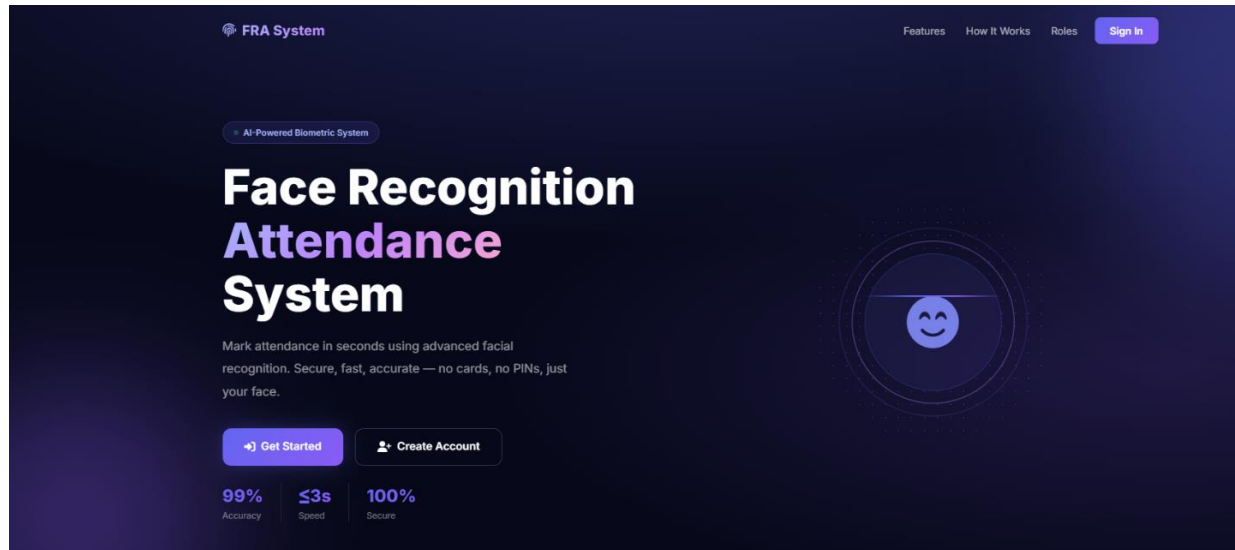


Figure 13 Main Interface

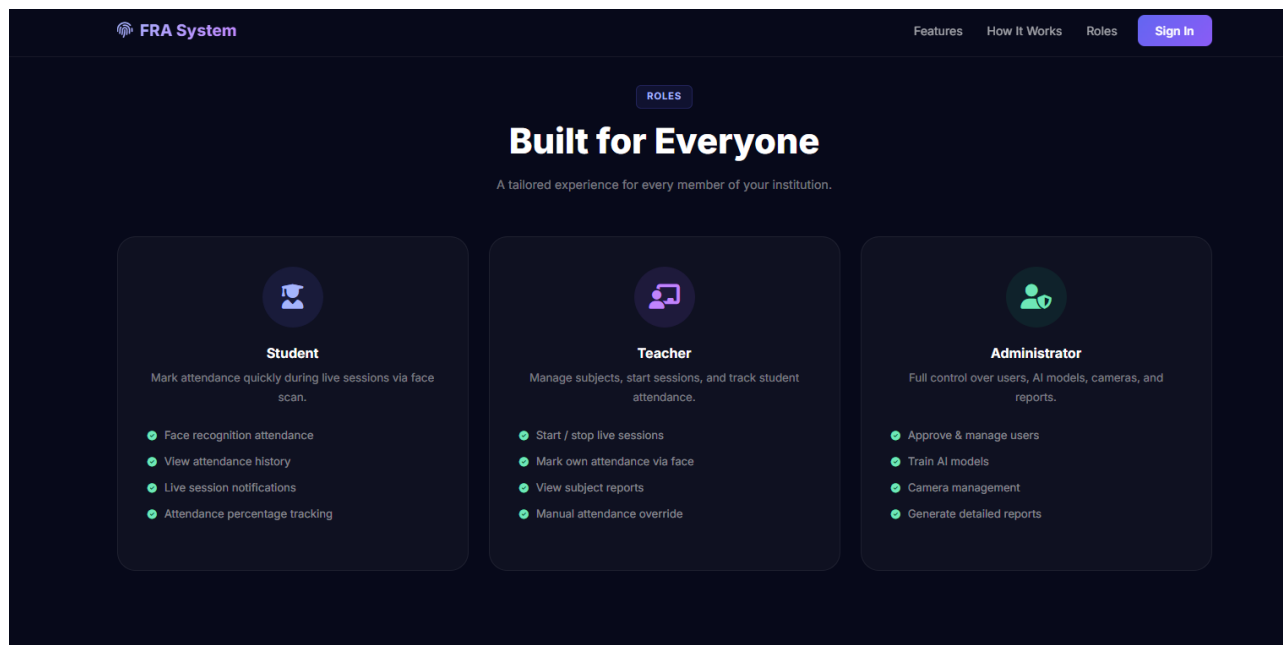


Figure 14 Roles in Interface

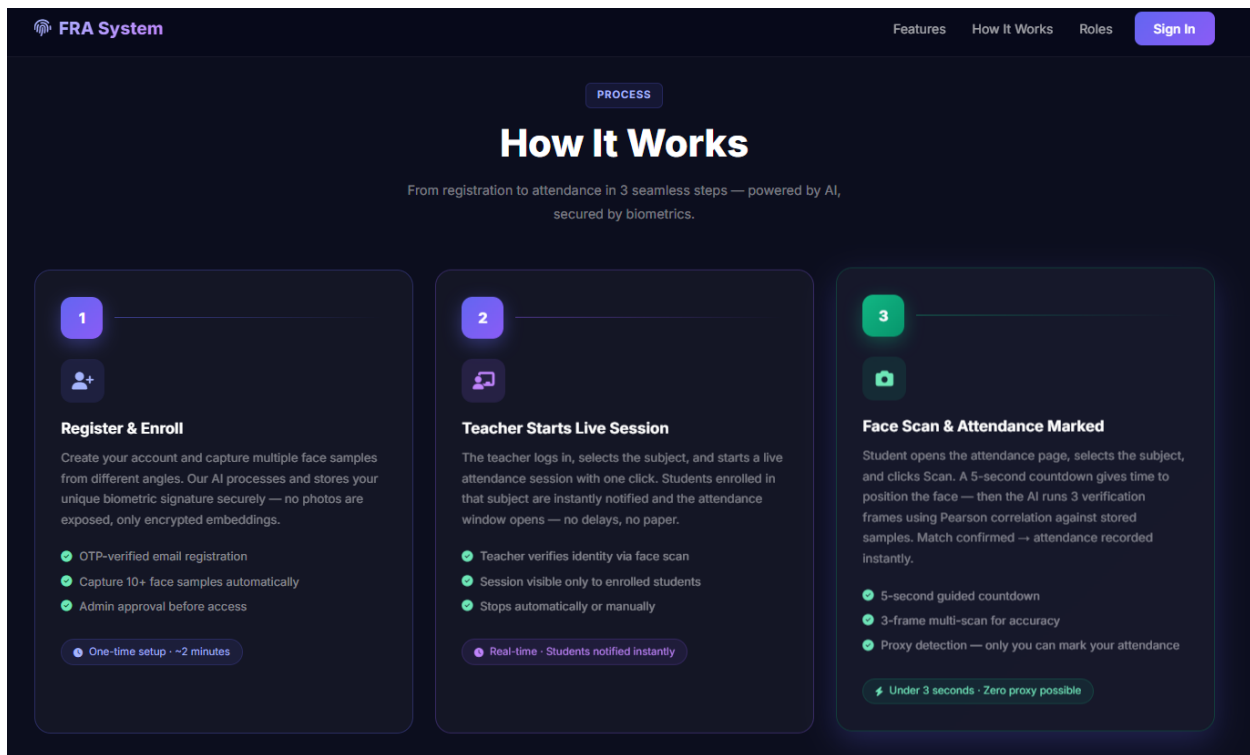


Figure 15 How Website Work

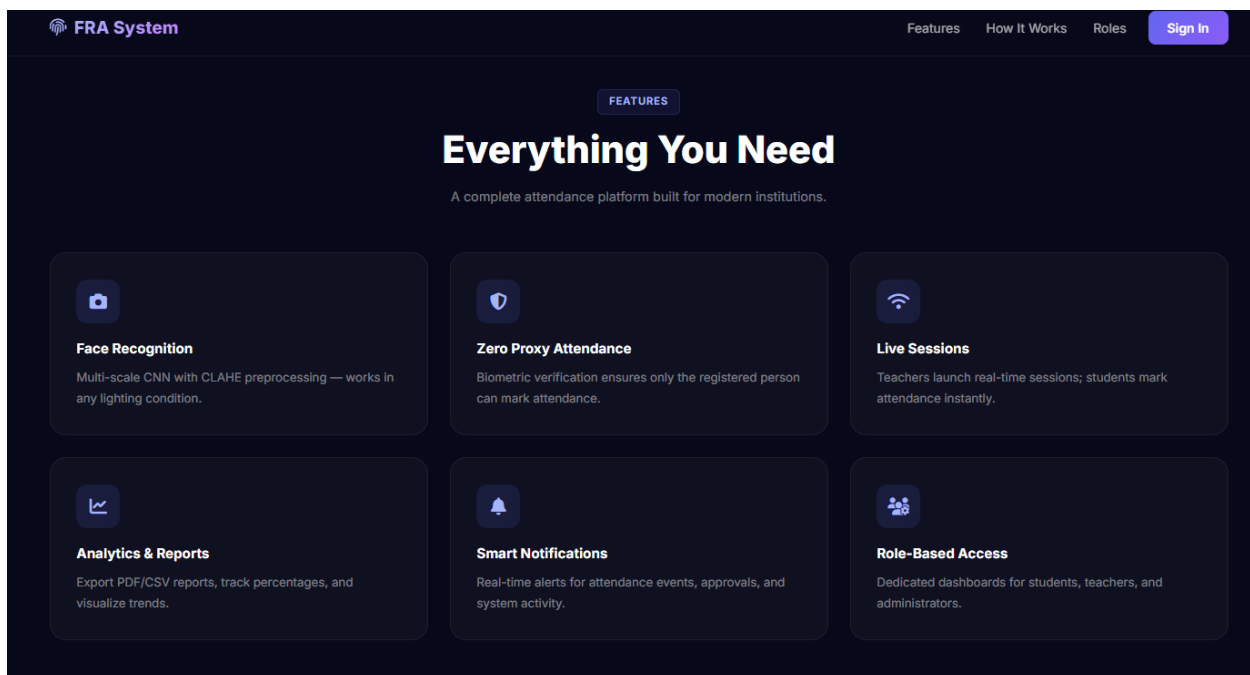


Figure 16 Features Of Website

6.2 Login & Authentication Screens:

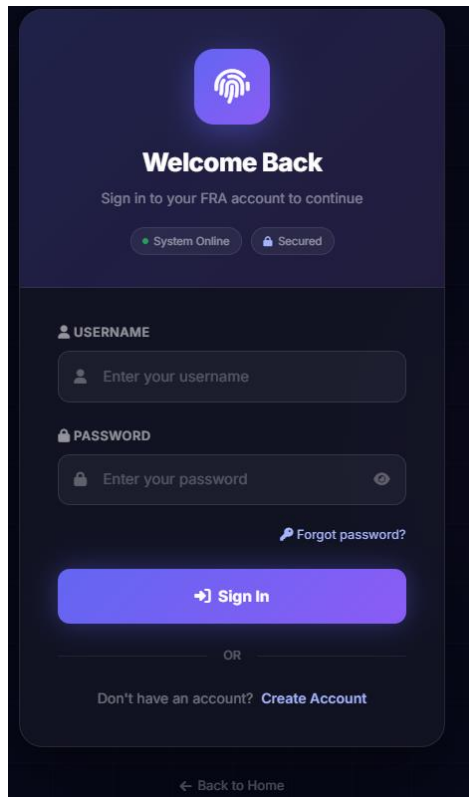


Figure 17 Login Page

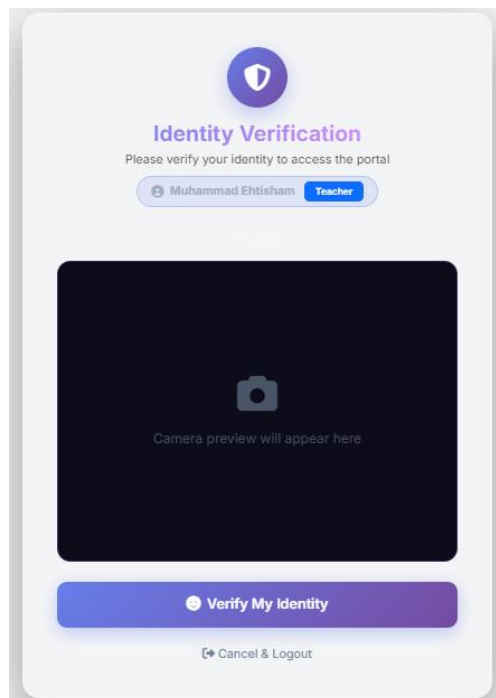


Figure 18 Teacher Login Verification

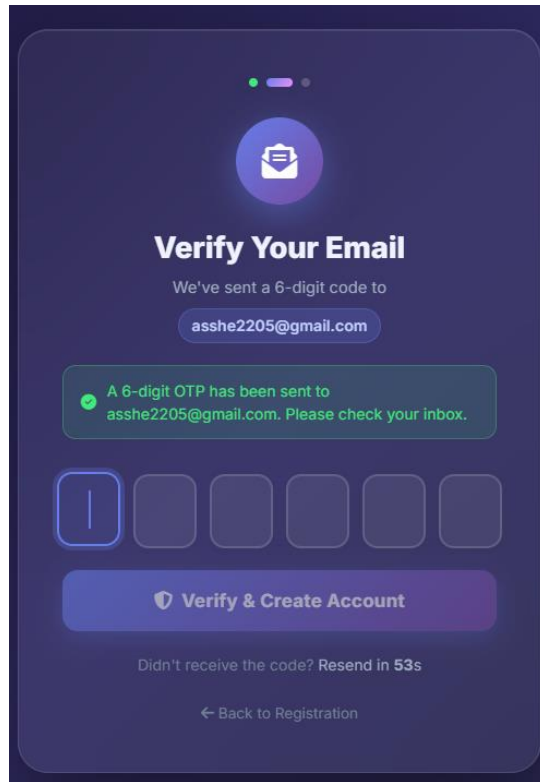


Figure 19 OTP Verification

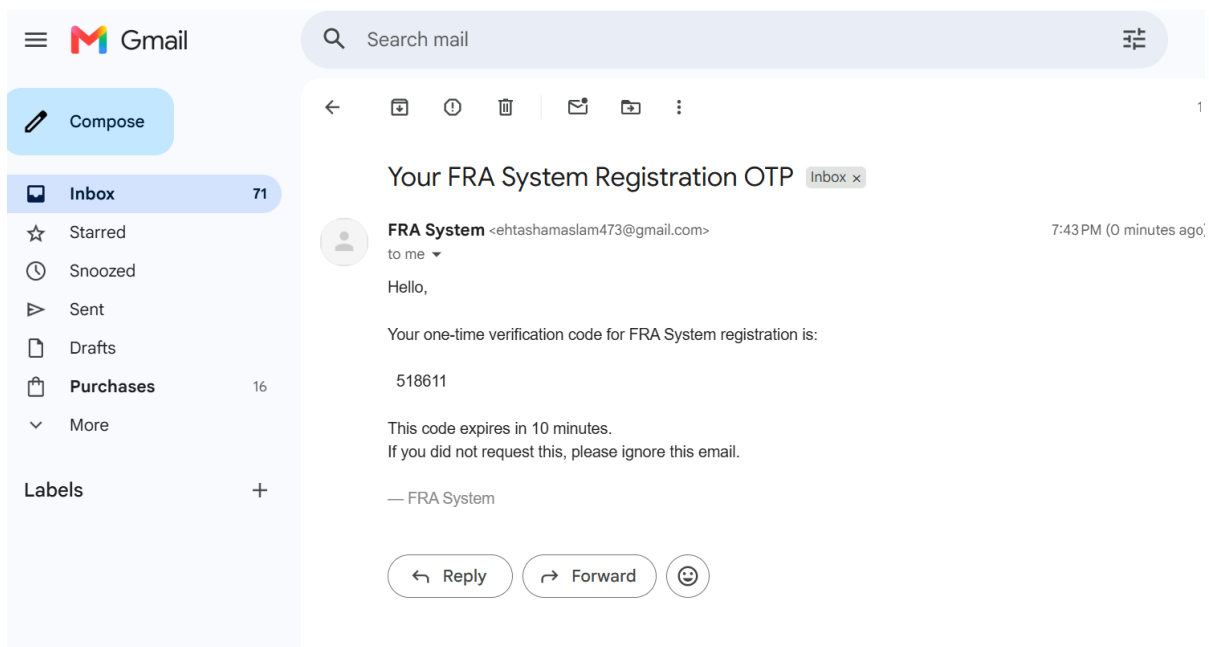


Figure 20 OTP VIA G Mail

6.3 Admin Dashboard:

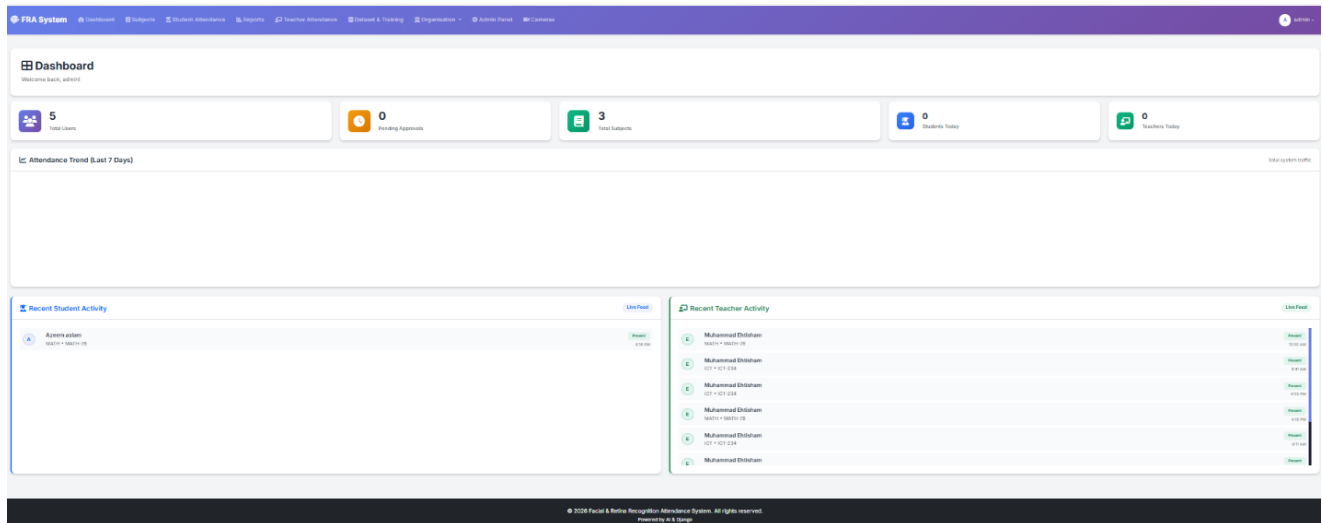


Figure 21 Admin Dashboard

- **Department, Subject and Section:**

Create Department

Department Name *

e.g. Computer Science

Short Code *

e.g. CS, BBA, EE

Used as a prefix throughout the system.

Description

Brief description (optional)

Cancel

✓ Create Department

Figure 22 Department Creation

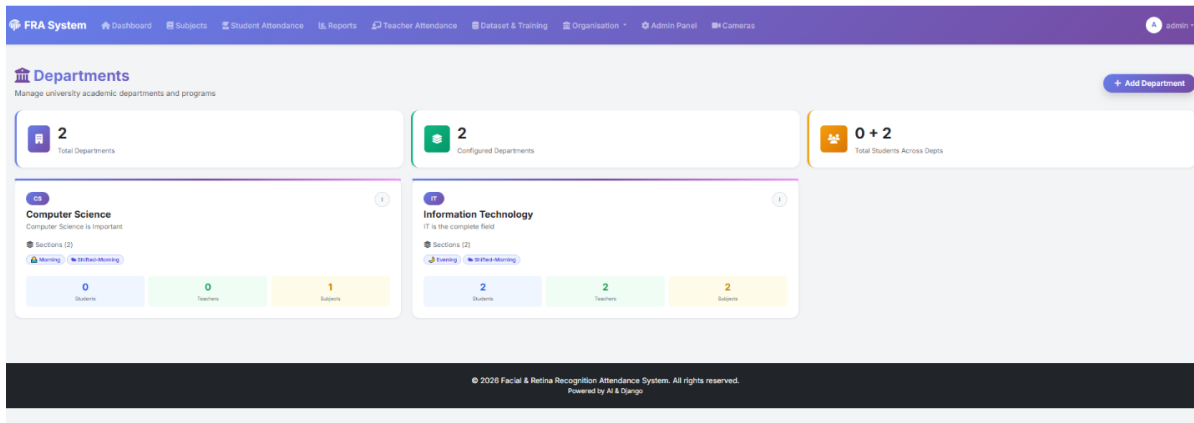


Figure 23 Departments Created by Admin

The 'Create Section' modal form contains two dropdown menus. The first is labeled 'Department *' and shows '-- Select Department --'. The second is labeled 'Section / Shift *' and shows '-- Select Shift --'. At the bottom, there are two buttons: 'Cancel' and '✓ Create Section'.

Figure 24 Section Creation

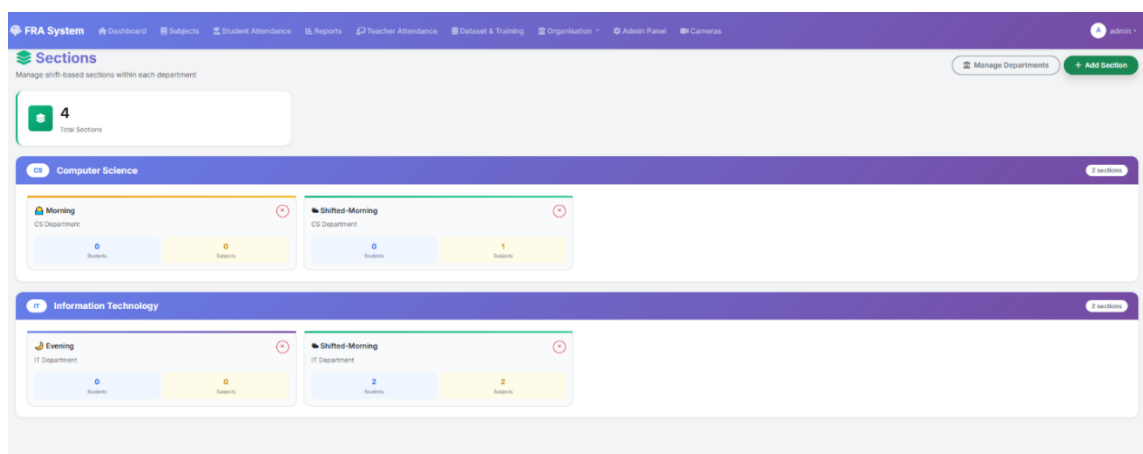
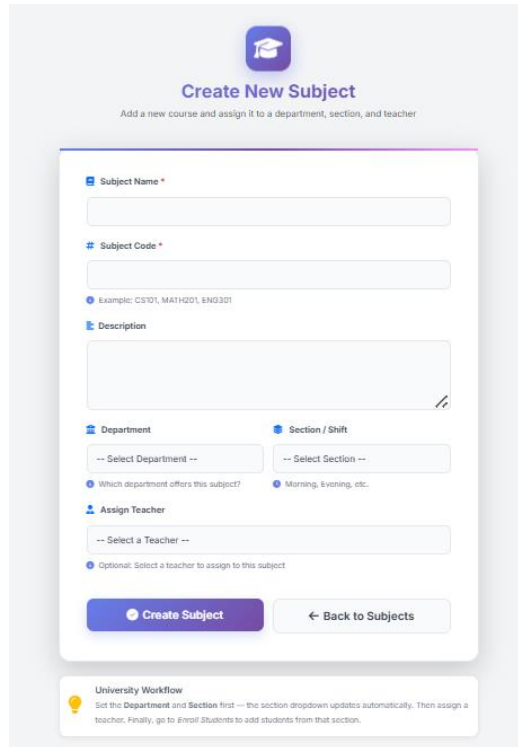


Figure 25 Section Created By Admin



Create New Subject
Add a new course and assign it to a department, section, and teacher

Subject Name *

Subject Code *

Example: CS101, MATH201, ENG301

Description

Department

Section / Shift

Which department offers this subject? ☐ Morning, Evening, etc.

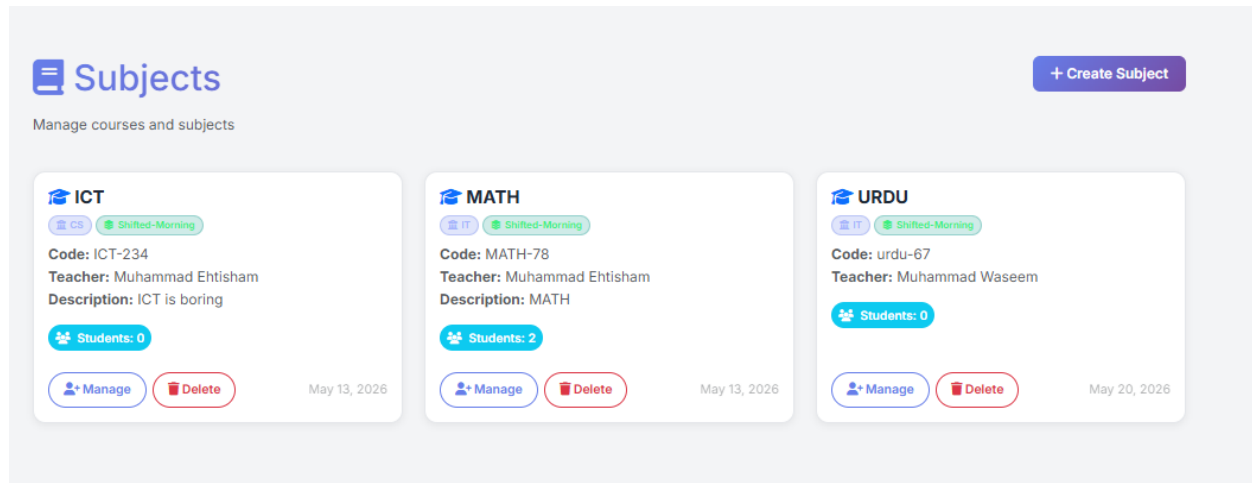
Assign Teacher

Optional: Select a teacher to assign to this subject

Create Subject **← Back to Subjects**

University Workflow
Set the Department and Section first — the section dropdown updates automatically. Then assign a teacher. Finally, go to Enroll Students to add students from that section.

Figure 26 Subject Creation



Subjects
Manage courses and subjects

+ Create Subject

ICT
CS Shifted-Morning
Code: ICT-234
Teacher: Muhammad Ehtisham
Description: ICT is boring
Students: 0
Manage **Delete** May 13, 2026

MATH
IT Shifted-Morning
Code: MATH-78
Teacher: Muhammad Ehtisham
Description: MATH
Students: 2
Manage **Delete** May 13, 2026

URDU
IT Shifted-Morning
Code: urdu-67
Teacher: Muhammad Waseem
Students: 0
Manage **Delete** May 20, 2026

Figure 27 Subject Created By Admin

- **Dataset and Model Training:**

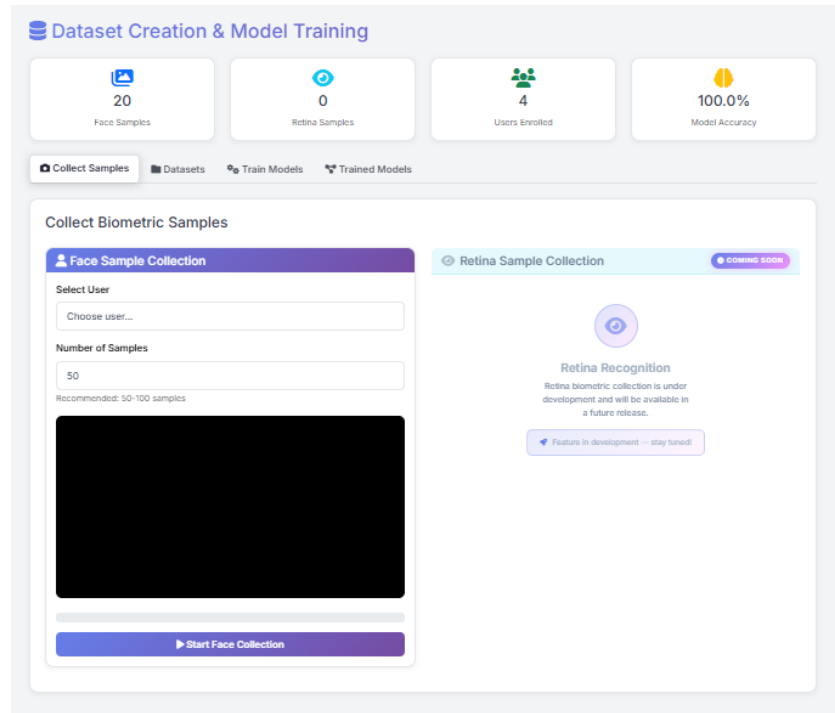


Figure 28 Collected Data Samples

Dataset Creation & Model Training

20 Face Samples 0 Retina Samples 4 Users Enrolled 100.0% Model Accuracy

Collect Samples **Datasets** Train Models Trained Models

Available Datasets [+ Create Dataset](#)

Name	Type	Samples	Status	Accuracy	Created	Actions
EMP-20260513-001	Face Recognition	5	Trained	100.0%	May 13, 2026	
STU-20260513-002	Face Recognition	5	Trained	100.0%	May 13, 2026	
STU-20260519-001	Face Recognition	5	Trained	96.67%	May 19, 2026	
EMP-20260520-001	Face Recognition	5	Trained	100.0%	May 20, 2026	

Figure 29 All Datasets

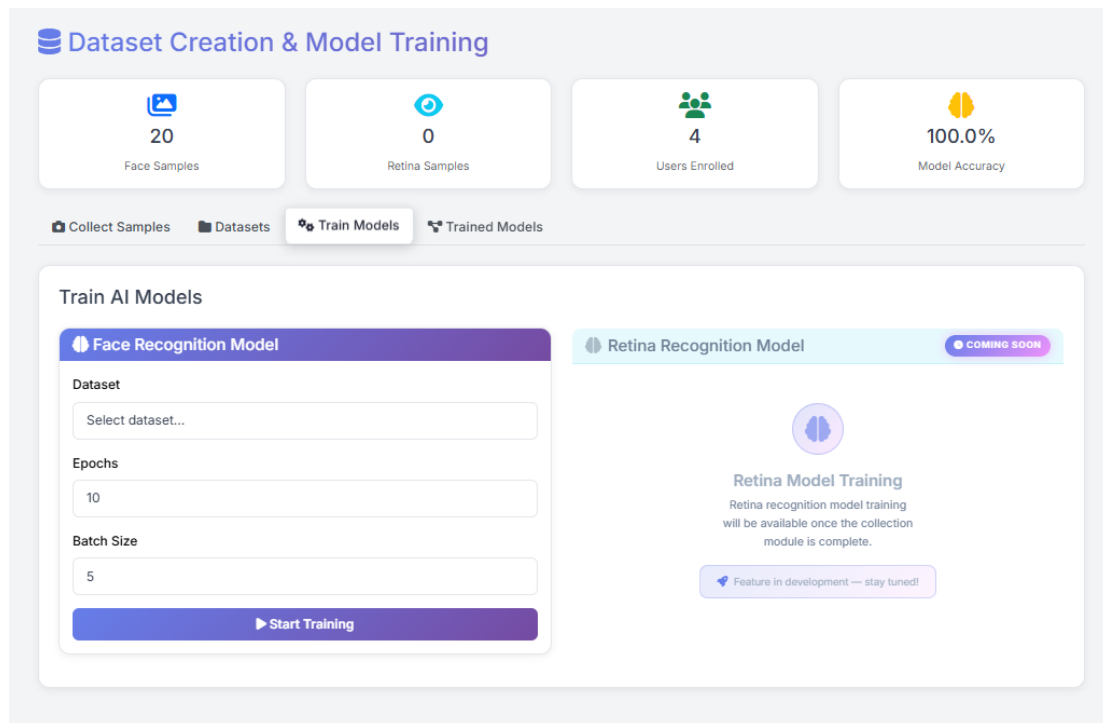


Figure 30 Dataset Training

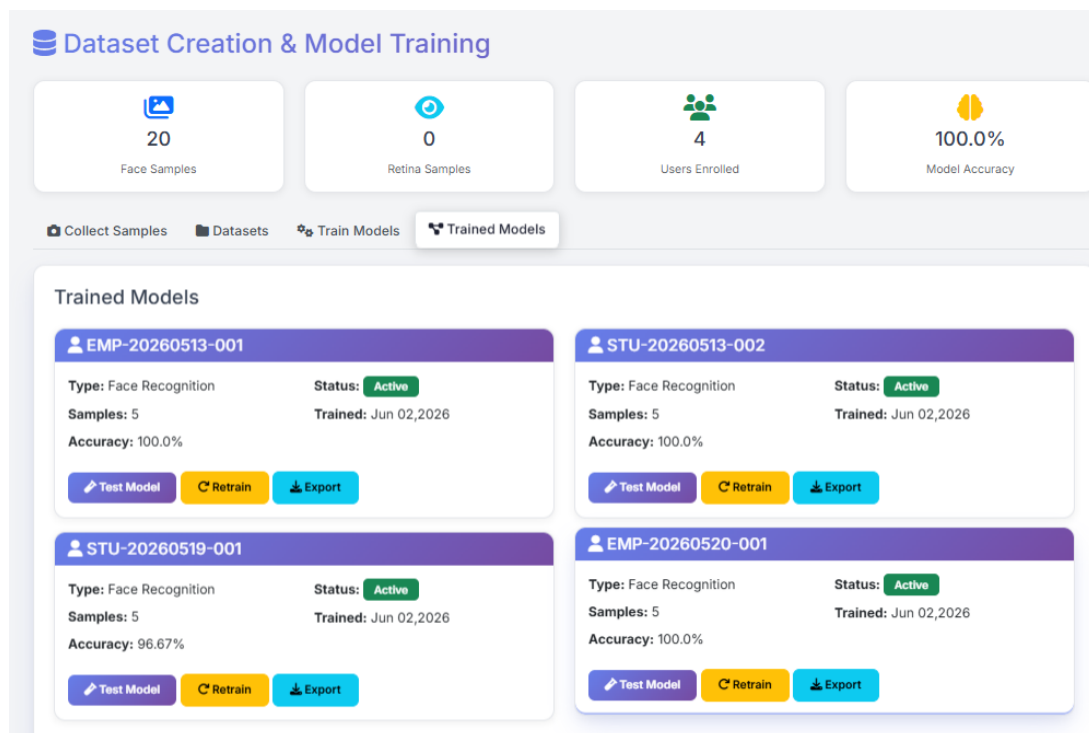


Figure 31 Trained Models

6.4 Teacher Interface:

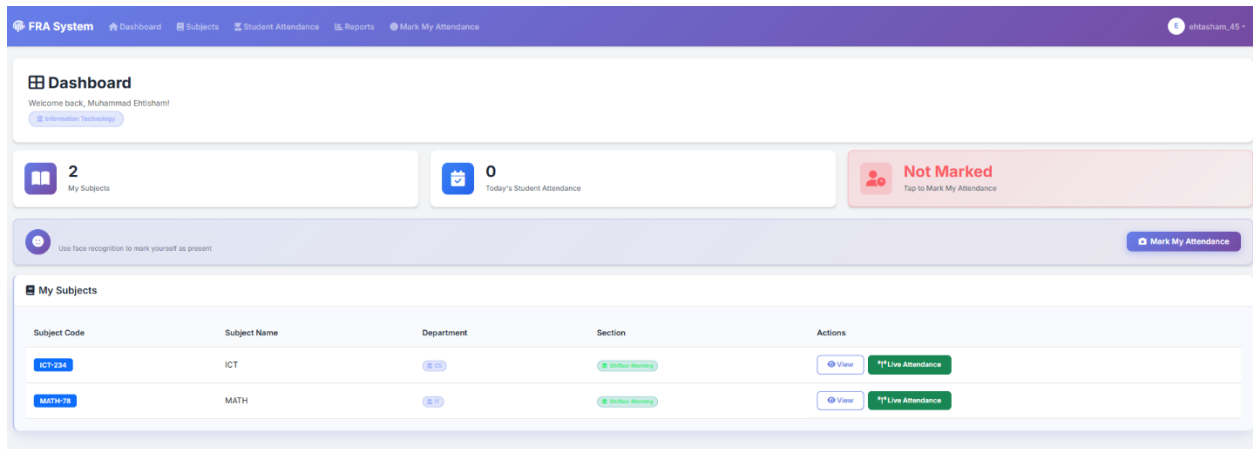


Figure 32 Teacher Dashboard

My Subjects				
Subject Code	Subject Name	Department	Section	Actions
ICT-234	ICT	ICT	Enrolled Morning	View Live Attendance
MATH-78	MATH	MATH	Enrolled Morning	View Live Attendance

Figure 33 Live Attendance Sessions

Student Attendance Management

[View Student History](#)

0
Today's Attendance

2
Total Students

0
Present Today

2
Absent Today

[Mark Attendance](#) [Edit Attendance](#) [Bulk Operations](#)

Mark Individual Attendance

Student

Select student...

Subject

Select subject...

Date

mm/dd/yyyy

Status

Present

Time

--:-- --

Notes (Optional)

Add any notes or reasons...

[Mark Attendance](#)

Figure 34 Student Attendance Manage by Teacher

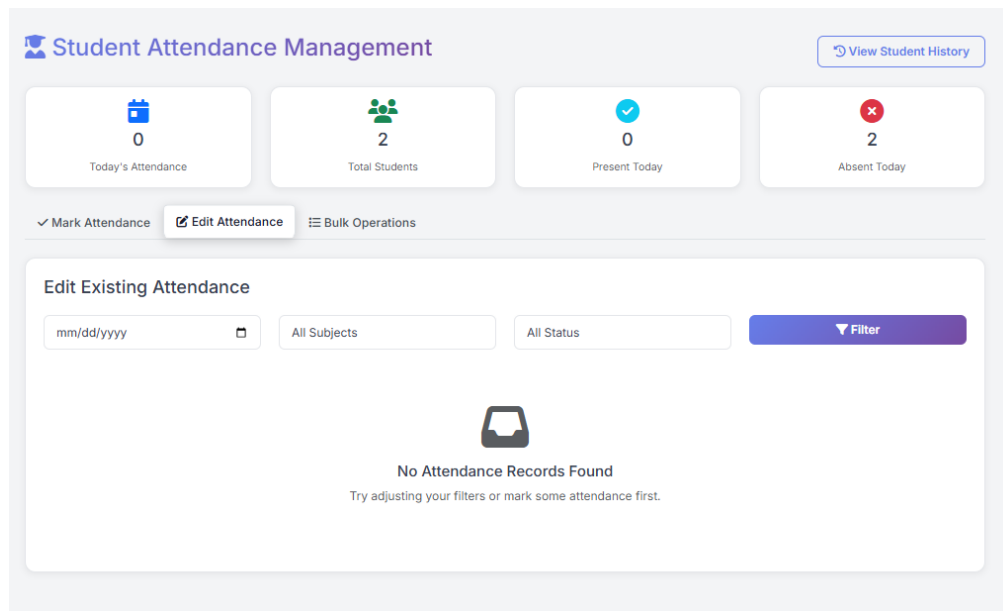


Figure 35 Edit Attendance

6.5 Student Interface:

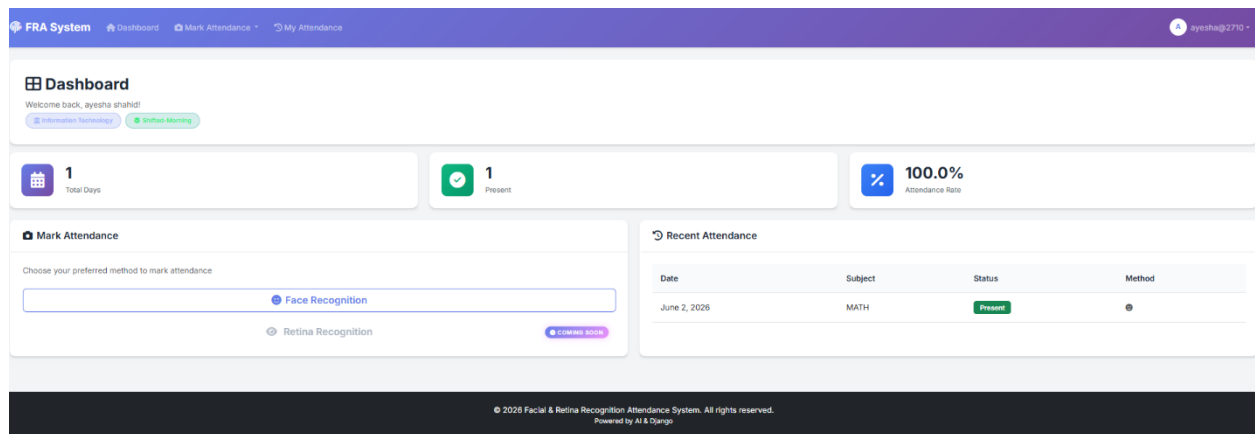


Figure 36 Student Dashboard

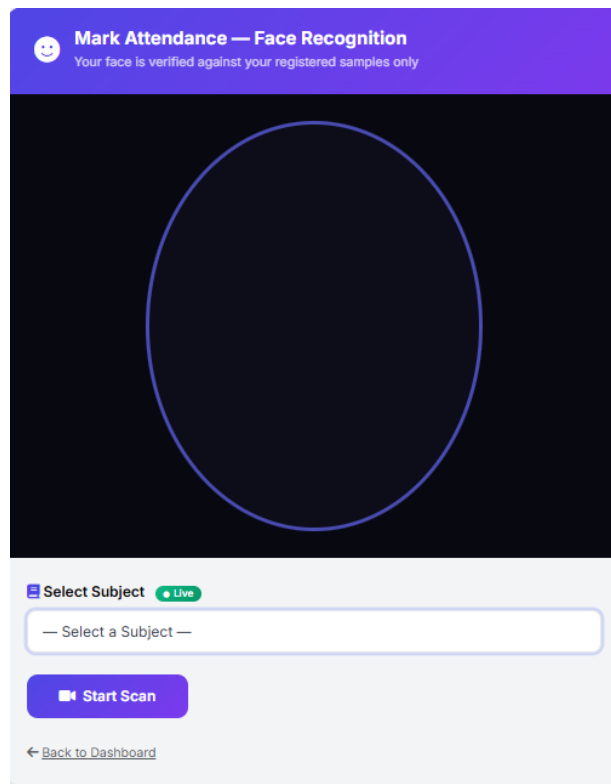


Figure 37 Student Attendance Mark Using Face Recognition

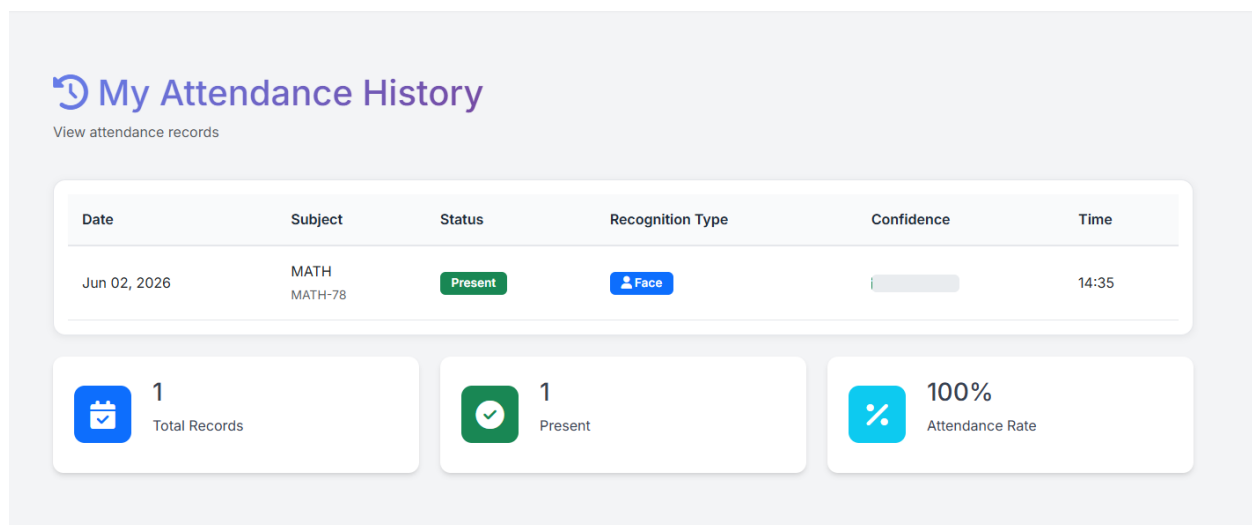


Figure 38 Student Attendance History

6.6 Reports & Audit Logs:

The screenshot shows a web form titled "Generate Attendance Report" with a subtitle "Configure filters and export a detailed attendance report in PDF or CSV format". The form is divided into several sections:

- Report Configuration:**
 - Report Type:** A dropdown menu set to "Daily". Below it, a note says "Choose daily, weekly, monthly or a custom date range".
 - Date Range:** Two date pickers for "Start Date" and "End Date", both showing "mm/dd/yyyy".
 - Organisation Filter:** A section with two sub-sections:
 - Department:** A dropdown menu set to "All Departments". A note below says "Leave blank for all departments".
 - Section / Shift:** A dropdown menu set to "— Select a department first —". A note below says "Select a department to enable".
 - Optional Filters:**
 - Subject:** A dropdown menu set to "— Select a department first —". A note below says "Select a department first to filter subjects".
 - Student:** A dropdown menu set to "All Students". A note below says "Leave blank to include all students".
 - Export Format:** Two radio buttons: "PDF" (selected) and "CSV".
- Footer:** A blue box with the text "Report downloads immediately. View past reports on the [Reports List](#) page." Below this are "Cancel" and "Generate Report" buttons.

Figure 39 Report Generation

The screenshot shows a web page titled "Attendance Reports" with a subtitle "View and generate attendance reports". In the top right corner, there is a "+ Generate Report" button. Below the title, there is a card for a generated report:

- Report Title:** "Daily Attendance Report — IT [Shifted-Morning] — MATH-78"
- Type:** "Daily"
- Period:** "Feb 25, 2026 - Jun 05, 2026"
- Generated:** "Jun 04, 2026 14:12"
- Actions:** "Download" (green button) and "Delete" (red button).

Figure 40 Report Generated

Recent System Logs				
Time	User	Action	Description	IP Address
Jun 02, 14:10	admin	Login	User admin logged in successfully	127.0.0.1
Jun 02, 14:08	ehtasham_45	Login	User ehtasham_45 logged in successfully	127.0.0.1
Jun 01, 17:10	ehtasham_45	Logout	User ehtasham_45 logged out	127.0.0.1
Jun 01, 17:10	ehtasham_45	Face Verification	ehtasham_45 passed face verification at login (best=0.76, matched 5/5 samples)	127.0.0.1
Jun 01, 17:08	ehtasham_45	Face Verification Failed	Login face-verify REJECTED for ehtasham_45 (best=0.84, matched 5/5 samples)	127.0.0.1
Jun 01, 17:08	ehtasham_45	Login	User ehtasham_45 logged in successfully	127.0.0.1
Jun 01, 17:07	admin	Logout	User admin logged out	127.0.0.1

Figure 41 System Logs